



Mémoire de Mission Professionnelle

Conception d'une application web intelligente pour la reconnaissance automatique des tuiles et le calcul des scores au Riichi Mahjong

UC D42 : Mission Professionnelle

Mastère européen Applications intelligentes et Big Data



Auteur : Henri TRAN

Introduction

À l'ère de la transformation numérique, de nombreux secteurs sont profondément impactés par l'essor du Big Data, de l'intelligence artificielle et des applications mobiles. Ces technologies permettent désormais d'automatiser des processus complexes, d'optimiser la prise de décision et d'améliorer significativement l'expérience utilisateur.

Le secteur du jeu vidéo, et plus particulièrement celui des contenus générés par les utilisateurs, s'inscrit pleinement dans cette dynamique d'innovation. Les plateformes modernes ainsi que les outils mis à disposition des créateurs sont de plus en plus performants et accessibles, permettant la conception d'expériences interactives variées. C'est dans ce contexte que s'inscrit l'expérience professionnelle réalisée lors d'un stage au sein de l'entreprise Arcade Royale, spécialisée dans la création de cartes pour le jeu Fortnite. Cette immersion a permis d'approfondir les connaissances en matière de processus de production ainsi que les méthodes de création propres à ce secteur d'activité.

Cependant, cette expérience a également mis en évidence certaines limites, notamment en ce qui concerne l'intégration de technologies avancées telles que l'intelligence artificielle, les bases de données ou les solutions d'automatisation. En effet, malgré la croissance du secteur, les possibilités techniques restent contraintes par les outils proposés par la plateforme. L'utilisation du langage de programmation Verse ainsi que l'hébergement des cartes sur des serveurs propriétaires limitent l'intégration de technologies externes. Ce constat met en lumière un potentiel encore largement inexploité en matière d'innovation technologique dans ce domaine.

Parallèlement, certains univers, notamment ceux des jeux traditionnels complexes, présentent des problématiques particulièrement intéressantes en matière d'automatisation et de gestion des données. Le Riichi Mahjong, variante japonaise du Mahjong, repose sur des règles riches et un système de calcul de score particulièrement complexe. Le calcul des points nécessite en effet la prise en compte de nombreux paramètres, tels que les combinaisons de tuiles, les conditions de

victoire ou encore le contexte de jeu, rendant cette tâche difficile pour les joueurs novices ou peu expérimentés.

Il apparaît dès lors pertinent de s'interroger sur l'apport des technologies récentes, en particulier l'intelligence artificielle appliquée à la vision par ordinateur, afin de répondre à ce type de problématique. Dans cette perspective, ce mémoire a pour objectif d'explorer la conception d'une application intelligente capable de reconnaître automatiquement des tuiles de Mahjong à partir d'une image, de calculer le score associé et de stocker les données de jeu dans une base de données afin de permettre l'analyse des performances des joueurs.

La problématique qui guidera ce travail est la suivante :
Comment concevoir une application intelligente reposant sur la vision par ordinateur afin d'automatiser la reconnaissance et le calcul des scores dans le Riichi Mahjong ?

Afin d'y répondre, ce mémoire s'articulera en trois parties. La première présentera le contexte professionnel ainsi qu'une analyse critique de l'existant ayant permis d'identifier un besoin technologique. La deuxième partie sera consacrée à la conception de la solution proposée, en détaillant les choix techniques et fonctionnels retenus. Enfin, la troisième partie abordera la mise en œuvre du projet, sa gestion ainsi qu'une analyse critique des résultats et des perspectives d'évolution.

Sommaire

Introduction **2**
Sommaire **4**
Partie 1 - Analyse du contexte professionnel et identification d'un besoin **5**
1.1 Présentation du secteur d'activité 5
1.3 Présentation du stage et des missions réalisées 6
1.4 Analyse critique de l'existant 7
1.5 Identification des limites et des besoins technologiques 7
1.6 Formulation de la problématique 8
Partie 2 – Conception d'une solution intelligente basée sur la vision par ordinateur **9**
2.1 Présentation du projet "Riichi Tool" 9
2.2 Étude de l'existant et analyse concurrentielle 10
2.3 Cahier des charges fonctionnel et non fonctionnel 13
2.4 Architecture technique de la solution 14
2.5 Choix des technologies 15
2.6 Modélisation des données et respect du RGPD 16
2.7 Conception du système d'intelligence artificielle 18
2.8 Développement de la logique du calcul des scores 33
Partie 3 – Mise en œuvre, gestion de projet et analyse critique **37**
3.1 Méthodologie de gestion de projet 37
3.2 Organisation de l'équipe et répartition des rôles 38
3.3 Planification du projet 38
3.4 Estimation des coûts et budget prévisionnel 39
3.5 Analyse des risques 40
3.6 Prototype et faisabilité technique 41
3.7 Analyse critique et limites du projet 45
3.8 Perspectives d'évolution 46
Conclusion **48**
Bibliographie **49**
Annexes **51**
Glossaire **53**

Partie 1 - Analyse du contexte professionnel et identification d'un besoin

1.1 Présentation du secteur d'activité

Le secteur du jeu vidéo constitue aujourd'hui l'un des domaines les plus dynamiques de l'industrie numérique, dépassant même ceux du cinéma et de la musique. Cette croissance continue est notamment portée par l'augmentation du nombre de joueurs à l'échelle mondiale ainsi que par les avancées technologiques.

Parmi les tendances majeures, les contenus générés par les utilisateurs (User Generated Content – UGC) occupent une place de plus en plus importante. Ce modèle permet aux joueurs de concevoir leurs propres expériences au sein des plateformes de distribution. Des environnements tels que Fortnite ou Roblox proposent des outils dédiés à la création, tout en offrant des systèmes de rémunération permettant aux créateurs de percevoir des revenus en fonction de l'engagement des joueurs (temps de jeu, fréquentation).

1.2 Présentation de l'entreprise Arcade Royale

Arcade Royale est une entreprise fondée en 2025 et spécialisée dans la création de cartes et d'expériences interactives au sein du jeu Fortnite. Elle évolue dans un environnement fortement concurrentiel, où la créativité, la capacité à proposer des expériences engageantes et la rapidité de production constituent des facteurs clés de succès.

L'activité principale de l'entreprise consiste à analyser les tendances du marché, puis à concevoir et développer des expériences jouables directement intégrées à Fortnite. L'objectif est de capter et de maintenir l'attention des joueurs en proposant régulièrement du contenu renouvelé.

L'organisation de l'entreprise repose sur une gestion de projet agile, combinant les méthodologies Scrum et Kanban. Des réunions quotidiennes (daily scrums) sont

organisées afin de suivre l'avancement des tâches, et des objectifs hebdomadaires sont définis pour assurer la mise à jour des expériences existantes ou le développement de nouveaux prototypes. Dans un environnement UGC, le manque de mises à jour régulières peut rapidement entraîner une baisse du nombre de joueurs et impacter la rentabilité des projets.

- **Organigramme actuel**



Figure 1 : Organigramme d'Arcade Royale

1.3 Présentation du stage et des missions réalisées

Au cours de ce stage, plusieurs missions ont été réalisées, principalement liées à la conception et à l'amélioration de cartes existantes. Ces missions incluaient l'ajout de nouveaux contenus, l'optimisation des mécaniques de jeu ainsi que l'apprentissage du langage Verse, utilisé dans l'écosystème UGC de Fortnite.

Ce stage a nécessité une grande polyvalence, impliquant à la fois des phases de recherche (analyse de concepts existants, veille sur les tendances), de développement (implémentation des mécaniques de jeu) et de conception visuelle (animations, design de l'environnement). Cette diversité de tâches reflète la nature multidisciplinaire de la création de contenus UGC.

Cette expérience a permis de développer des compétences techniques liées aux outils de création de Fortnite, mais également des compétences organisationnelles, notamment en gestion de projet et en travail collaboratif. Le rythme soutenu du secteur impose en effet une forte réactivité afin de respecter les délais, les retards pouvant avoir un impact direct sur la performance des cartes.

1.4 Analyse critique de l’existant

Malgré les nombreux avantages offerts par les outils de Fortnite, plusieurs limites ont été identifiées au cours de cette expérience. En effet, les possibilités de création sont restreintes à l’environnement fermé de la plateforme, ce qui limite fortement l’intégration de technologies externes.

Le langage de programmation Verse, bien qu’adapté à certains usages, ne permet pas d’exploiter pleinement des technologies avancées telles que l’intelligence artificielle, qui nécessitent généralement des outils et bibliothèques externes. Par ailleurs, l’hébergement des cartes sur les serveurs de Fortnite empêche l’utilisation de bases de données externes, limitant ainsi les possibilités de stockage et d’analyse des données.

Ces contraintes réduisent les possibilités d’automatisation et d’innovation, et mettent en évidence un écart entre les capacités offertes par les technologies actuelles et leur exploitation réelle dans cet environnement.

1.5 Identification des limites et des besoins technologiques

Les limites observées soulignent un besoin d’évolution vers une meilleure intégration des technologies avancées, telles que les bases de données, l’analyse de données et l’intelligence artificielle. Ces technologies jouent aujourd’hui un rôle central dans de nombreux secteurs et constituent des leviers majeurs d’innovation.

Dans ce contexte, il apparaît pertinent d’explorer des domaines dans lesquels ces technologies restent encore sous-exploitées, avec pour objectif d’améliorer l’efficacité, l’accessibilité et les fonctionnalités proposées aux utilisateurs.

Les jeux traditionnels complexes, tels que les échecs, le jeu de Go ou encore le Mahjong, représentent un terrain d’application particulièrement intéressant. Ces jeux présentent des problématiques adaptées à l’utilisation de technologies avancées, notamment en matière d’analyse et d’automatisation.

Les avancées dans ce domaine sont déjà visibles, comme en témoigne la victoire d'une intelligence artificielle dans le jeu de Go face à un joueur professionnel de haut niveau. De plus, plusieurs intelligences artificielles performantes existent aujourd'hui pour les échecs. En revanche, le Mahjong reste encore relativement peu exploité sous cet angle, malgré l'existence de solutions comme Suphx.

1.6 Formulation de la problématique

Au regard des éléments précédemment analysés, il apparaît pertinent de proposer une solution innovante intégrant des technologies d'intelligence artificielle, en particulier dans le domaine de la vision par ordinateur.

Si les jeux comme les échecs ou le Go disposent déjà de nombreuses solutions basées sur l'intelligence artificielle, le Mahjong reste encore peu développé dans ce domaine, notamment en ce qui concerne les applications exploitant la reconnaissance visuelle.

Je peux ainsi donc poser la problématique suivante :

"Comment concevoir une application intelligente reposant sur la vision par ordinateur afin d'automatiser la reconnaissance et le calcul des scores dans le Riichi Mahjong ?"

Partie 2 – Conception d’une solution intelligente basée sur la vision par ordinateur

2.1 Présentation du projet “Riichi Tool”

Le projet porte sur la conception d’un site web dédié au Riichi Mahjong intégrant un système de vision par ordinateur ainsi qu’une base de données permettant de stocker les mains jouées et les statistiques des joueurs.

Le Riichi Mahjong est un jeu complexe, car de nombreux paramètres doivent être pris en compte afin de calculer correctement le score d’une main. En effet, selon les combinaisons de tuiles, le type de victoire (Ron ou Tsumo), le vent dominant, le vent du joueur, le statut de dealer, la présence de Dora ainsi que plusieurs autres éléments, le nombre de points obtenu peut fortement varier.

L’ensemble de ces paramètres constitue une véritable barrière à l’entrée pour les joueurs débutants ou novices, notamment lorsqu’ils ne disposent pas d’un joueur expérimenté capable de réaliser rapidement le calcul du score.

L’objectif du site est donc de permettre aux utilisateurs de prendre en photo leur main victorieuse, d’y ajouter les informations complémentaires nécessaires, telles que le vent dominant, le statut de dealer, l’appel du Riichi ou encore les Dora, puis de laisser l’application reconnaître automatiquement les tuiles grâce au système de vision par ordinateur intégré.

Le score final de la main est ensuite calculé automatiquement. Le système détaille les Han, les Fu ainsi que les Yaku détectés, et indique également le niveau de score correspondant lorsque cela s’applique, comme Mangan, Haneman, Baiman, Sanbaiman ou Yakuman.

Les mains analysées peuvent ensuite être enregistrées dans une base de données afin de constituer un historique des mains gagnantes et de produire des statistiques détaillées sur l’ensemble des performances du joueur.

Cette solution permet ainsi de répondre à la problématique posée tout en améliorant l'accessibilité du Riichi Mahjong et en proposant une expérience enrichie grâce à l'analyse des données.

2.2 Étude de l'existant et analyse concurrentielle

Il est essentiel d'analyser les applications déjà présentes sur le marché afin d'identifier leurs forces, leurs faiblesses ainsi que les opportunités d'amélioration permettant de proposer une solution plus pertinente.

Plusieurs applications et sites spécialisés existent déjà, mais la majorité d'entre eux ne permettent pas l'utilisation de la caméra ou d'une photographie, et beaucoup ne prennent pas en charge les règles spécifiques du Riichi Mahjong japonais. De plus, certaines applications sont uniquement disponibles sur l'App Store d'Apple et non sur le Google Play Store, tandis que d'autres ne sont pas adaptées aux utilisateurs francophones ou anglophones, ce qui limite leur accessibilité.

Tout d'abord il y a Mahjong Camera et JongCamera qui sont respectivement sur l'App Store d'Apple et sur le Google Play Store. JongCamera ne propose que les règles du style Hong Kong ou Taiwanais alors que Mahjong Camera propose aussi le Riichi. De plus, la détection des tuiles avec la camera de JongCamera présente beaucoup d'erreurs ce qui montre que le model de reconnaissance n'as pas été assez bien entrainé.

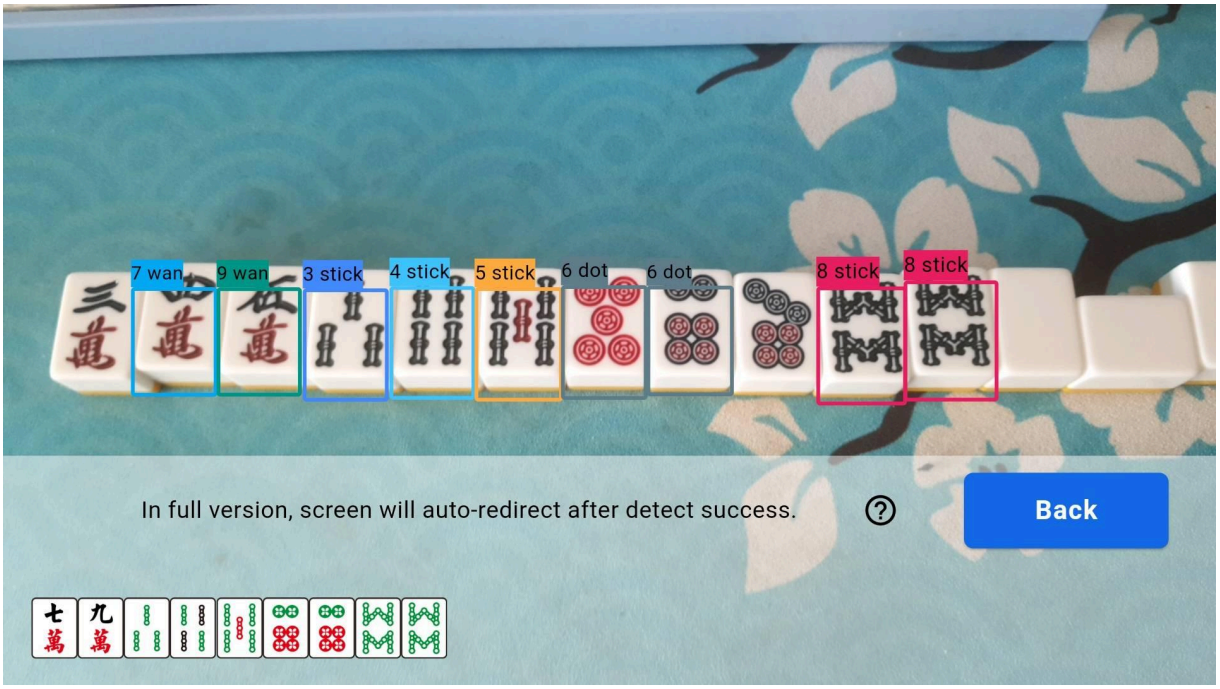


Figure 2 : Essai de la détection de JongCamera

On peut voir avec la Figure 2 que certaines tuiles ne sont pas reconnus comme le 3 de caractère, le 7 de cercle et les dragon blancs. Mais aussi que d'autres sont mal reconnus comme le 4 et 5 de caractère qui deviennent un 7 et un 9, ou le 5 rouge de cercle qui devient un 6 de cercle.

L'application Mahjong Camera quant à lui permet de scanner des mains de Riichi et permet même de sauvegarder des parties entières en scannant chaque main gagnante de la partie. On peut observer qu'une grande partie des utilisateurs sont satisfait de l'application et son utilité pour les joueurs plus débutant.

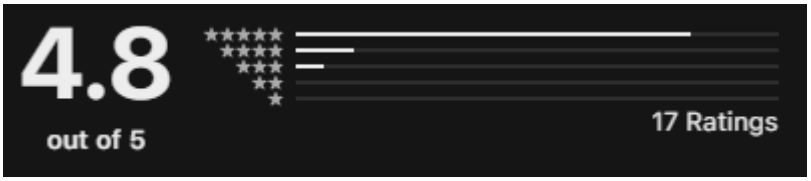


Figure 3 : Note de Mahjong Camera sur l'Apple Store



Figure 4 : Avis positif d'un débutant à propos de Mahjong Camera

Les autres applications disponibles comme Riichi Calc sur le Google Store n'utilise pas la vision par ordinateur et force l'utilisateur à entrer tuile par tuile les mains gagnantes.

Il présente une note très bonne malgré le manque de fonctionnalité comme la reconnaissance des tuiles ou un historique des mains et des statistiques possible.



Figure 5 : Note de Riichi Calc sur le Google Play Store

Finalement, les quelques sites web permettant de calculer le score du Riichi Mahjong se présentent comme Riichi Calc et n'ont pas d'implémentation de détection ni de sauvegarde des mains pour un historique ou des analyses de statistiques. On peut par exemple citer Mahjong Calc Livewing ou Agari.

Cette étude montre qu'il existe un manque réel sur le marché, notamment du côté des applications Android et des sites web, concernant les solutions intégrant à la fois la détection automatique des tuiles, l'historique des mains et l'analyse statistique des performances.

Lorsque la détection visuelle existe, sa précision reste souvent insuffisante pour une utilisation fiable. Riichi Tool se différencie ainsi par trois objectifs principaux : une

meilleure précision de reconnaissance, la conservation de l'historique des parties et l'exploitation des statistiques utilisateur, permettant de proposer une solution plus moderne, plus accessible et plus complète.

2.3 Cahier des charges fonctionnel et non fonctionnel

Afin de concevoir ce projet, il est nécessaire d'établir un cahier des charges regroupant les besoins fonctionnels, les contraintes techniques ainsi que les perspectives d'évolution.

Attentes fonctionnelles

Le projet doit répondre aux fonctionnalités essentielles suivantes :

- insertion d'une photo de main de Mahjong
- reconnaissance automatique des tuiles présentes sur l'image
- possibilité de corriger manuellement les erreurs éventuelles de détection
- saisie des informations complémentaires nécessaires au calcul final (Ron, Tsumo, Dora, Riichi, dealer, vent dominant, etc.)
- calcul automatique du score final
- affichage détaillé du calcul de la main (Han, Fu et Yaku détectés)
- possibilité d'enregistrer la main dans une base de données
- historique des mains gagnantes
- statistiques détaillées sur les performances du joueur

Contraintes non fonctionnelles

Plusieurs contraintes doivent également être respectées afin de garantir la qualité du projet :

- précision de détection proche de 90 % ou supérieure
- interface simple, intuitive et rapide à utiliser
- temps de réponse faible pour garantir une bonne expérience utilisateur
- conformité avec le RGPD pour la gestion des données personnelles
- accessibilité depuis un navigateur web sans installation complexe

Perspectives d'évolution

Le projet pourra évoluer afin d'attirer un plus grand nombre d'utilisateurs et d'améliorer l'engagement sur le long terme.

Parmi les évolutions possibles :

- implémentation d'un système de recherche d'utilisateurs permettant de consulter les statistiques d'amis ou d'autres joueurs, avec gestion des autorisations de confidentialité
- ajout de plusieurs langues afin d'améliorer l'accessibilité internationale
- mise en place d'un système de gamification avec quêtes, récompenses et éléments cosmétiques de profil
- ajout d'autres règles de Mahjong afin d'élargir la cible utilisateur
- évolution du frontend vers React pour une architecture plus scalable et potentiellement multiplateforme

2.4 Architecture technique de la solution

Le projet est divisé en plusieurs parties distinctes afin de garantir une architecture modulaire et une séparation claire des responsabilités.

Le frontend est chargé de l'interface utilisateur et permet :

- l'insertion de l'image de la main
- la saisie des informations complémentaires
- l'accès à l'historique des mains
- l'accès aux statistiques détaillées du joueur

Le backend est lui-même composé de trois modules principaux :

- le modèle d'intelligence artificielle chargé de la reconnaissance visuelle des tuiles
- le moteur de calcul du score basé sur les règles standards du Riichi Mahjong
- l'application principale permettant de relier le frontend, la base de données et les différents traitements métier

Les informations d'une main sont d'abord traitées par le module de vision par ordinateur, puis transmises au calculateur de score. Une fois le résultat affiché, l'utilisateur peut choisir de sauvegarder ou non la main dans la base de données.

La base de données permet de stocker les informations des utilisateurs, les mains enregistrées ainsi que les statistiques globales.

Cette architecture favorise la maintenance, la modularité et l'évolution future du projet. Chaque composant étant séparé, il devient plus simple d'identifier l'origine d'un problème, d'améliorer une fonctionnalité spécifique ou de faire évoluer l'application vers une version plus avancée.

2.5 Choix des technologies

Les technologies retenues ont été choisies selon plusieurs critères : le coût, les performances, la simplicité de développement et la facilité de maintenance.

Pour le frontend, le site a été développé en HTML, CSS et JavaScript natif afin de relier l'interface utilisateur au backend. Ce choix permet une grande simplicité de développement, un temps de mise en place rapide ainsi que de bonnes performances pour un projet de taille moyenne. Contrairement à des frameworks comme React ou Vue, cette approche nécessite moins de configuration initiale et reste parfaitement adaptée à un prototype fonctionnel.

L'intelligence artificiel utilise le modèle YOLOv11 (You Only Look Once) afin de détecter les tuiles de Mahjong. Ce modèle à été choisi pour ses performances de détection d'objets et sa rapidité d'inférence une fois entrainé mais aussi pour sa facilité d'utilisation.

L'annotation du dataset a été réalisée manuellement à l'aide de LabelImg sur des photographies prises personnellement, permettant de constituer un premier jeu de données d'apprentissage composé actuellement d'environ 80 images annotées.

Python est utilisé pour l'entraînement du modèle ainsi que pour l'ensemble de la logique d'intelligence artificielle. Le modèle final est exporté au format ONNX (Open

Neural Network Exchange), permettant une meilleure portabilité ainsi qu'une inférence plus rapide dans l'application.

Flask a été retenu pour le backend afin de créer une API légère assurant la communication entre le frontend, le moteur de calcul, l'intelligence artificielle et la base de données. Cette solution permet une architecture simple, flexible et facilement maintenable.

Enfin, MySQL est utilisé comme base de données pour stocker les mains analysées, les statistiques ainsi que les informations utilisateurs. La communication entre Python et MySQL est assurée par MySQL Connector. Ce choix se justifie par la fiabilité de MySQL, sa simplicité d'utilisation ainsi que sa bonne adaptation aux besoins relationnels du projet.

Dans une version future plus avancée, une migration vers PostgreSQL pourrait être envisagée afin de bénéficier d'une gestion encore plus robuste des transactions et d'une meilleure conformité aux exigences avancées des systèmes de production.

2.6 Modélisation des données et respect du RGPD

La base de donnée se repose sur trois tables principales :

- Utilisateur
- Partie
- Main

La table utilisateur contiens toutes les données liée au compte du joueur ainsi que sont identifiant qui sera la clé primaire qui permet de relier le joueur aux différentes tables. Il est nécessaire de respecter les exigences du Règlement Général sur la Protection des Données (RGPD) car notre projet implique la récolte et le traitement de certaines données des utilisateurs notamment leurs informations de connection, les éventuelles photos des mains de Mahjong, leur historique de partie ou même leurs statistiques globales.

Le principe de minimisation des données sera appliqué en ne récoltant que les informations strictement nécessaire au fonctionnement du site. Les données

sensibles ne seront en aucun cas demandé par le site ou par un utilisateur de l’équipe de développement.

De plus, l’utilisateur sera informé de quels données seront récoltés, quels sont leur utilisation, leur durée de conservation, et leur droit d’accès, de modification ou de suppression de leur données personnelles. L’utilisateur devra accepter d’avoir lu ces termes afin de pouvoir accepter au site.

Un système de suppression de compte sera implémentée avant la version finale du site afin de respecter les règles strictes de la RGPD.

Les données seront sécurisés par le chiffrement des mot de passe, par l’accès du site uniquement par des connections sécurisés et aussi par une limitation stricte de l’accès aux données n’autorisant que le développeur en chef d’y avoir accès.

La table Partie permettra de regrouper les différentes mains d’une même session. Elle contiendra un identifiant de partie et une date fin de pouvoir y accéder facilement. De plus, elle sera lié aux joueurs qui sont inclus dans la partie et contiendra le nombre de joueur, les règles de jeu, les différentes mains joués et la progression de score durant la partie.

Enfin, la table Main contiendra toutes les informations d’une main que ce soit les tuiles reconnues, les informations contextuelles, ainsi que le détail du score et le resultat final. Elle sera relié ou non à une Partie et à son Utilisateur.

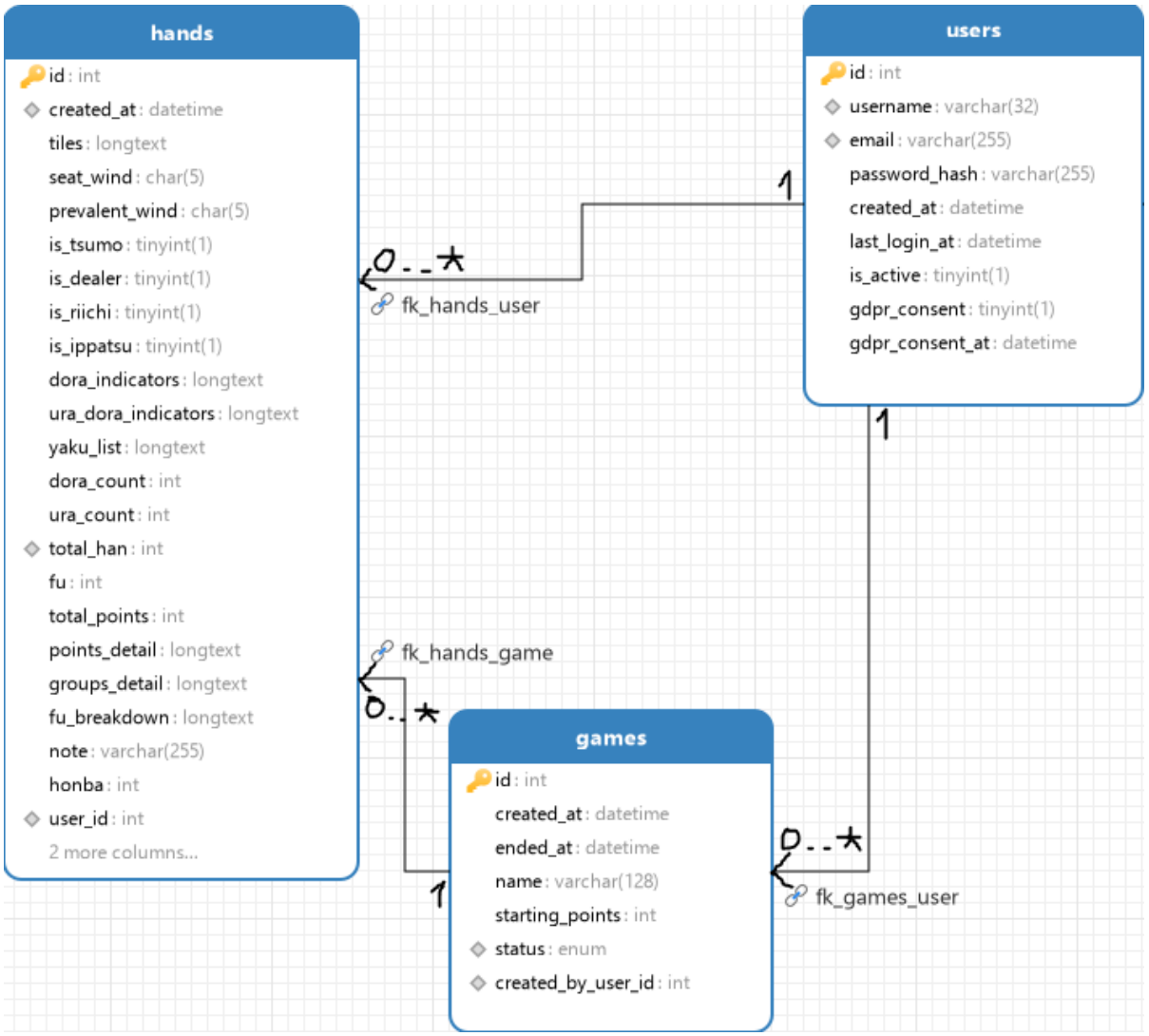


Figure 6 : Relations entre les tables de la base de donnée

2.7 Conception du système d'intelligence artificielle

Le système innovant du projet se repose sur l'intégration d'un système de reconnaissance de tuiles de Mahjong automatique avec la vision par ordinateur à partir d'une photo.

Le paradigme choisi est celui du Deep Learning, spécialement dans celle appliqué à la détection d'objets par image. Le modèle de YOLOv11 sera utilisé dans ce projet pour sa capacité à reconnaître plusieurs objets sur une seule image et de pouvoir déterminer leur position sur l'image. De plus sa capacité d'inférence est supérieur à d'autres modèles comme Faster R-CNN qui est plus précis sur ses analyses.

Le modèle YOLOv11 utilise moins de mémoire et obtient de meilleurs résultats de précision comparé au modèle YOLOv8 qui est très similaire.

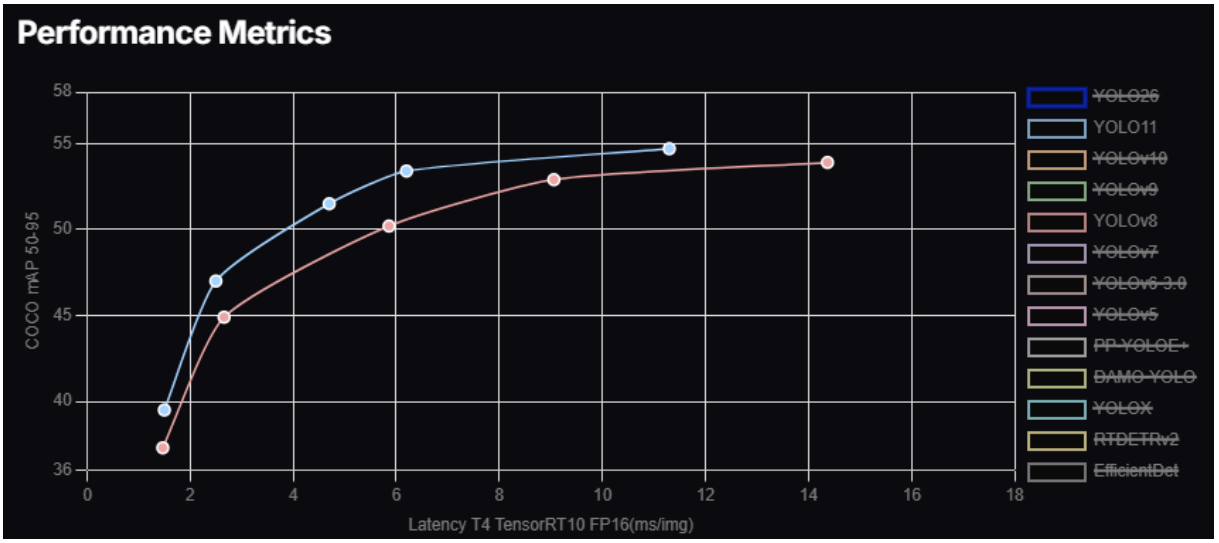


Figure 7 : Graphique de comparaison des performance de YOLO8 et YOLO11

Le fonctionnement du système est divisé en plusieurs étapes cruciales :

1. Acquisition de l'image :

Les images sont prises avec des mains de 14+ tuiles de Mahjong. Plus de 80 photos ont été prises manuellement afin d'avoir des images de tuiles avec différents conditions possible tels que l'angle des tuiles, des fonds variés ou une luminosité différente. Un set de Mahjong Riichi basique avec le design des tuiles classique est utilisée afin de pouvoir être utilisé par le plus grand monde possible. D'autres design pourront être ajoutés au model au fur et à mesure que les demandes d'utilisateurs augmentent. Chaque photo contiennent de réelles combinaisons de main possible afin d'habituer le modèle aux positions des tuiles en condition réel.

2. Etiquetage des images :

LabelImg est ensuite utilisé afin d’assigner des rectangles de chaque image à un label qui correspond à chaque tuile de jeu.
Le Riichi Mahjong utilise 37 tuiles différentes avec 4 exemplaires chacun sauf le 5. Il se sépare en 3 familles de chiffres de 1 à 9 avec le 5 qui aura 3 exemplaire classique et 1 exemplaire en rouge qui s’appelle Aka Dora ou Red Five.

Les noms des familles sont les suivantes :

- Manzu : la famille des charactères, qui sont représentés par les charactères chinois de 1 à 9.
- Pinzu : la famille des ronds.
- Souzu : la famille des bambous.

Nous avons ensuite les 4 différents vents qui sont les points cardinaux dans l’ordre :

- Ton : est
- Nan : sud
- Shaa : ouest
- Pei : nord
-

Et finalement les 3 dragons :

- Haku : dragon blanc
- Hatsu : dragon vert
- Chun : dragon rouge



Figure 8 : Liste de chaque tuile de Riichi Mahjong

La liste des étiquettes utilisé sera donc :

Manzu : 1m, 2m, 3m, 4m, 5m, 6m, 7m, 8m, 9m, 0m (qui correspond au 5 rouge)
Pinzu : 1p, 2p, 3p, 4p, 5p, 6p, 7p, 8p, 9p, 0p
Souzu : 1s, 2s, 3s, 4s, 5s, 6s, 7s, 8s, 9s, 0s
Vents : E, S, W, N
Dragons : Haku (blanc), Hatsu(vert), Chun(rouge)

Avec cette liste d'étiquette, nous pouvons l'importer dans LabelImg et ainsi encadrer et étiquetter chaque tuiles de chaque image de la dataset.

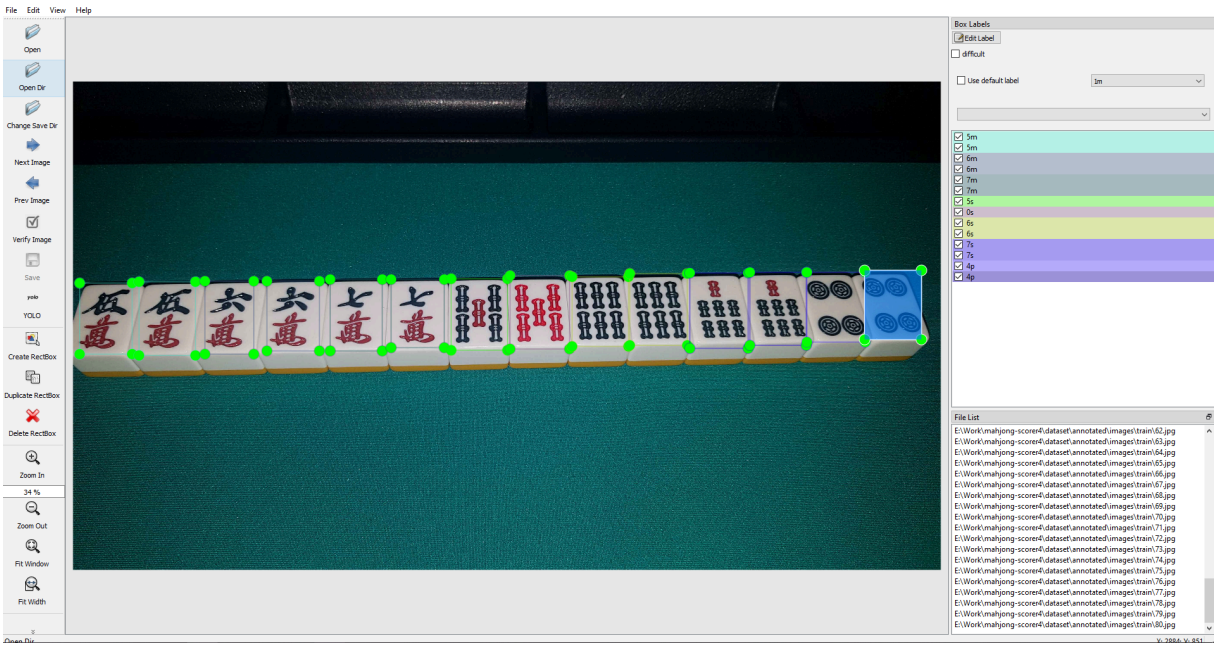


Figure 9 : Processus d'étiquettage des tuiles des images dans LabelImg

Le format de sortie des etiquettes sera prêt directement à être utilisé par le modèle YOLO. En effet, chaque image étiquetté nous donnera un fichier .txt avec le même nom que l'image afin de pouvoir être relié entre eux et contiendra sur chaque ligne le label de la tuile suivi de quatres valeurs permettant de retrouver le rectangle.

Les quatres valeurs sont les suivantes dans l'ordre :

- La coordonnée x du centre du rectangle normalisé entre 0 et 1 relatif à la taille de l'image sur la largeur.
- La coordonnée y du centre du rectangle normalisé entre 0 et 1 relatif à la taille de l'image sur la hauteur.
- La largeur du rectangle normalisé entre 0 et 1 relatif à la taille de l'image sur la largeur.
- La hauteur du rectangle normalisé entre 0 et 1 relatif à la taille de l'image sur la hauteur.

Ces valeurs sont normalisé et relatives à la taille de l'image car cela permet de rester valide même si l'image se fait redimensionner. Ce qui est le cas avec YOLO qui redimensionnent les images à 640x640 pixel avec les settings standards.

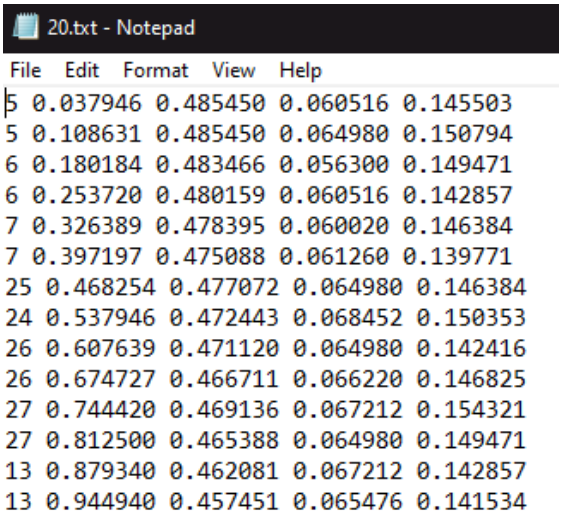


Figure 10 : Example de fichier sorti par LabelImg pour l'image de la Figure 8

3. L'entrainement du modèle

Cette partie est cruciale dans la conception du système car c'est celle qui permet au système d'apprendre à reconnaître des tuiles dans des photo entières et aussi à les différencier les uns des autres.

Différents paramètres sont disponibles quand on veut lancer une session d'entrainement du modèle YOLO :

- Epochs : Les epochs ou époques en francais, correspondent à un cycle entier d'entrainement où le modèle parcourera l'entièreté des images de la dataset. Il ne faut pas avoir un nombre d'epoch trop faible car le modèle ne sera pas assez bien entrainé. Mais il ne faut pas non plus avoir un nombre trop élevé car il pourra faire du overfitting, ce qui empêche le modèle de prédire de bons résultats en condition réels car il sera trop entrainé sur nos images et sera imprécis si nos photos réelles ont des minimes différences comme la lumière ou l'angle.

- Batch : Correspond au nombres d’images utilisé par groupe d’image. Une epoch entiere correspond au nombre total d’image du dataset divisé par le nombre de batch. Donc par exemple pour un dataset de 64 images, et un batch de 8, on aurait 64/8 = 8 donc 8 groupes de 8 images qui correspond bien a un total de 64 images de la dataset et donc d’une epoch complète. Les images sont à chaque fois mélangés au hasard au début de chaque epoch afin de ne pas avoir le même ordre.

- Model : Le choix du modèle de YOLOv11 correspond à différents types d’utilisation finale de l’IA et de la puissance du CPU (processeur) ou GPU (carte graphique) du serveur.
  5 types de modèles sont disponibles : Nano , Small, Medium, Large et XLarge.
  Plus le modèle est grand, plus il demandera de RAM à l’utilisation et ralentira l’inférence au moment de l’utilisation. Le Nano et Small utilisent un CPU tandis que à partir du Medium, un GPU est recommendé.

Nos paramètres d’entrainement pour environ 80 photos étiquettés seront :

- Epochs : 100
- Batch : 8
- Model : Small

Ce nombre d’epoch nous permet d’être sûr de ne pas avoir un nombre d’epoch trop faible, et nous pouvons vérifier dans les résultats de l’entrainement si on a de l’overfitting.

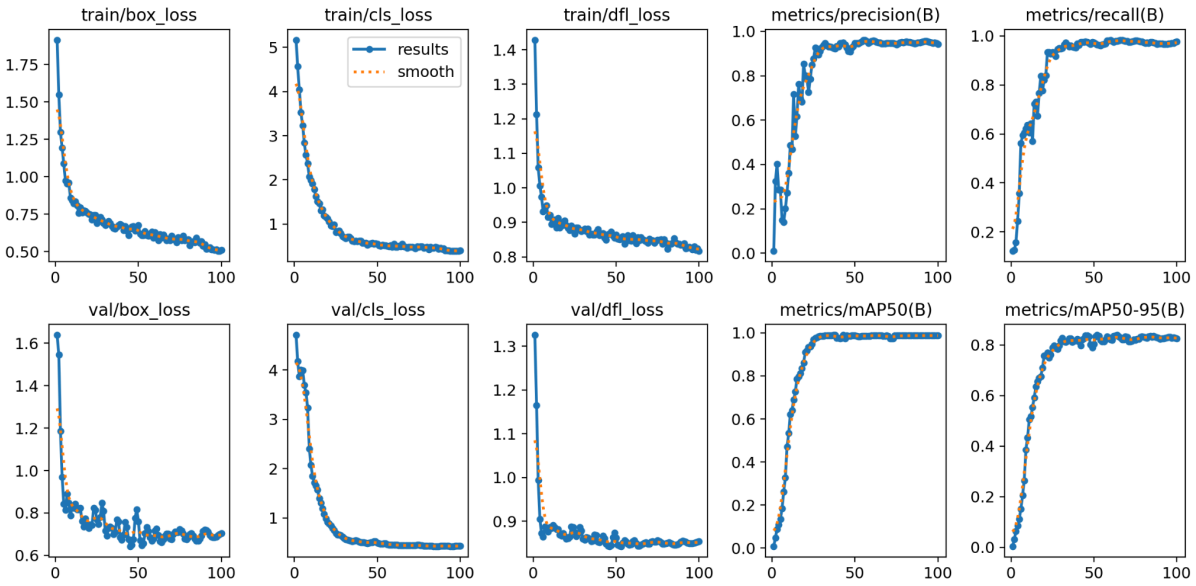


Figure 11 : Graphiques des résultat obtenus de l'entrainement avec les paramètres annoncés

On peut observer sur l'image plusieurs graphiques avec chacun un nom différent.

- Box_loss correspond à quel point le modèle se trompe sur la position des tuiles sur l'image.
- Cls_loss correspond à quel point le modèle se trompe sur le label de la tuile existante à l'intérieure de chaque rectangle.
- Dfl_loss correspond à l'imprécision des contours des rectqngles.
- Train correspond aux résultats de l'entrainement avec 80% des images de la dataset.
- Val correspond aux résultats des tests effectués avec le reste des 20% d'images de la dataset qui ne sont jamais utilisés dans l'entrainement et qui permettent ainsi de vérifier après chaque epoch à quel point il fonctionnerait sur une image inconnue.
- Recall correspond au pourcentage de tuiles que le modèle arrive à trouver sur les photos.
- mAP50 correspond à la moyenne de précision à un seuil d'acceptation précis qui décide si le rectangle avec le label est accepté comme correct ou non. Si on mets un seuil a 50%, lorsque le modèle est sûr à plus de 50% que le rectangle et le label correspondent à une tuile sur l'image, alors elle sera compté comme correct et acceptée dans le résultat final de détection. Et donc

si le modèle n’est sûr qu’a 45% de la tuile, alors elle ne sera pas compté comme correct car elle sera en dessous du seuil d’acceptation.

- Le mAP50-95 correspond à la moyenne total de chaque seuil de 5 entre 50 et 95.

Afin de voir si il y a de l’overfitting, les graphiques qui nous intéressent sont le train cls_loss et le val cls_loss en principal. Tant que les deux valeurs continuent de descendre plus on avance dans les epochs, il n’y a pas d’overfitting. Dans notre cas, nous pouvons observer qu’aux environs de l’epoch 30, nous atteignons un plateau qui continue de descendre lentement. En nous intéressant aux valeurs exactes dans le fichier results.csv resultant de l’entrainement avec le modèle YOLO, nous pouvons voir :

Epoch	Train cls_loss	Val cls_loss
80	0.48484	0.42993
90	0.44214	0.43279
95	0.39246	0.42601
100	0.39702	0.43144

Cela nous montre que jusqu’à 95 epochs , notre modèle continue de s’améliorer puis commence à se dégrader après jusqu’a l’epoch 100. Cependant 5 epochs n’est pas suffisant pour créer de l’overfitting donc nous somme dans un cas fiable.
De plus, on pourrait améliorer nos résultats et réduire les chances d’overfitting en ajoutant des photos étiquettés en plus dans le dataset.

4. Analyse des résultats de l’entrainement

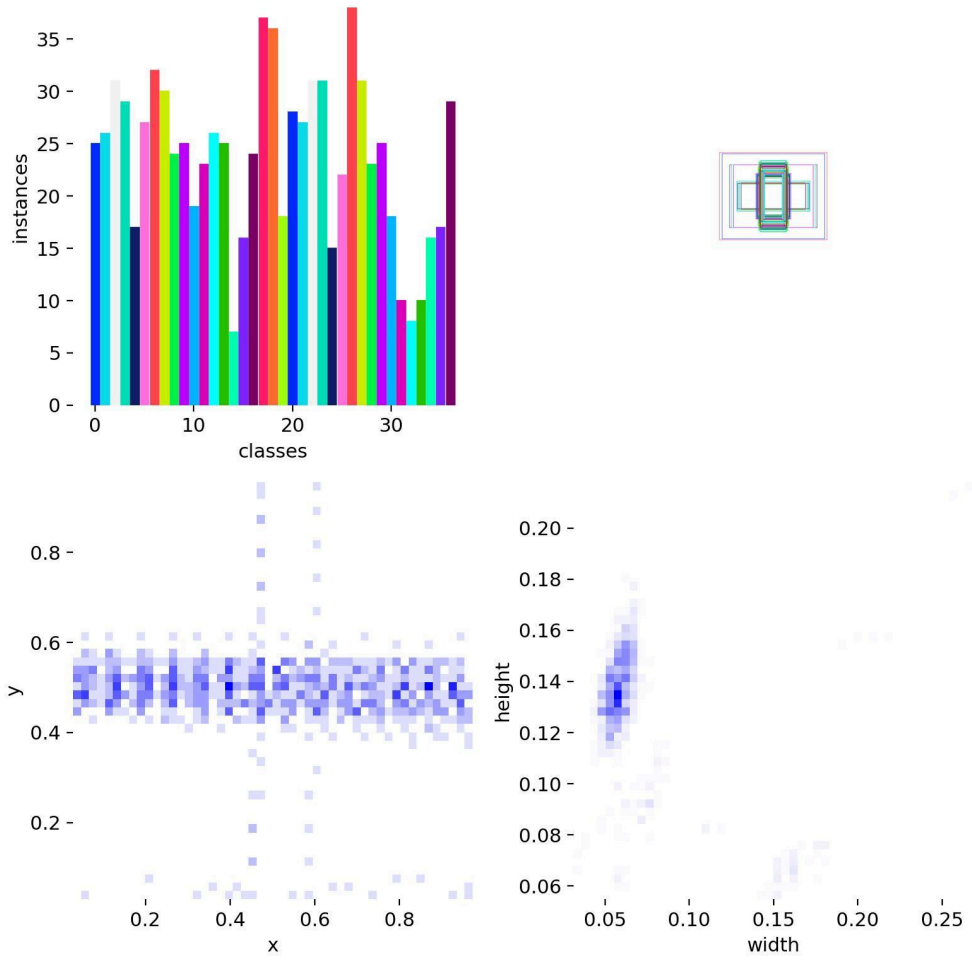


Figure 12 : Graphiques des informations sur les labels avant l'entrainement

La figure 12 nous montre 4 différents graphiques :

- Le premier en haut à gauche correspond à la répartition du nombre de tuiles correspondant à chaque label à partir de toutes les images du dataset.
  On peut observer un manque d'instances de certaines tuiles du dataset qui nous indique qu'il faudrait ajouter plus d'images dans le dataset afin d'avoir une répartition plus égale et avoir plus de data pour ces tuiles.

- Le deuxième en haut à droite nous montre la forme et taille visuelle des tuiles qu'on à étiquettés. Ici, leur forme est généralement uniforme ce qui montre que nos données sont assez constants.

- Le troisième graphique en bas à gauche correspond à la position des centres des tuiles par rapport a l'image. On voit que la pluspart des points sont vers y = 0.5 ce qui montre qu'une grande partie des images du dataset ont été prises alignés vers le milieu de la photo. Cela peut être un point négatif du modèle car il n'aura pas d'expérience sur des tuiles qui serait de travers ou en diagonal.

- Le dernier graphique nous indique la taille générale des tuiles par rapport à la taille de l'image. On observe une concentration des points autour de 0.05 en largeur et 0.14 en hauteur ce qui correspond bien à une uniformité de la taille des tuiles.

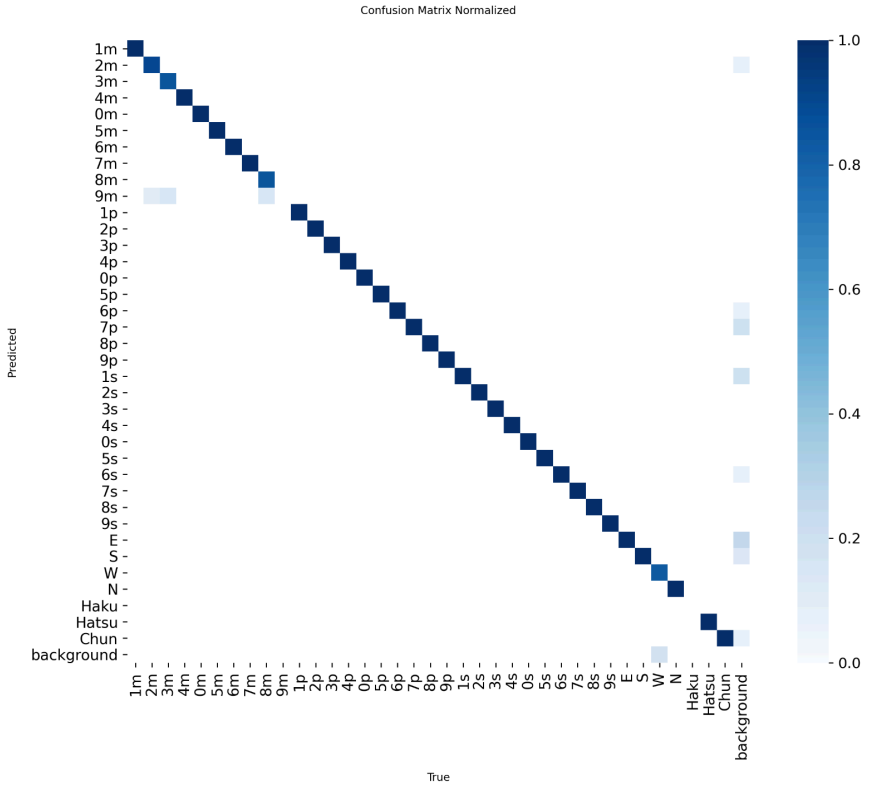


Figure 13 : Matrice de confusion normalisé du modèle

La figure 13 nous montre une grille des 37 tuiles avec en vertical , les labels des tuiles que le modèle prédit, et en horizontal les vrai labels des tuile. Donc plus un carré est foncé, plus le modèle réussi sa prédiction. Un graphique parfait montrerai

donc une diagonale complètement bleu foncé.

On peut observer qu'une grande partie des tuiles ont un très bon taux de prédiction mis à part le 9 de charactère qui se confond avec les autres tuiles de la même famille, et le Haku (Blanc) qui n'apparaît pas car la valeur est répartie sur toutes les autres tuiles comme la tile est totalement blanche.

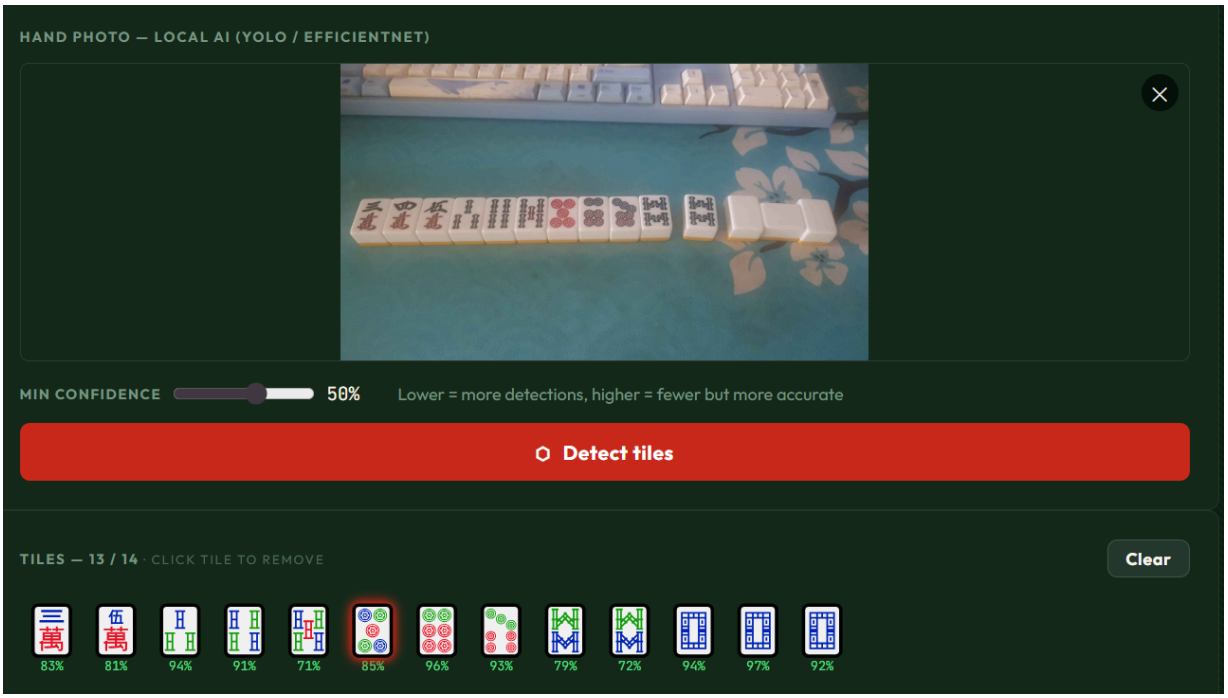


Figure 14 : Image de la détection des Haku (Blanc) avec le site

On observe sur la figure 14 que malgré le manque de couleur sur la matrice de confusion, les tuiles Haku de la photo ont été reconnues parfaitement ce qui montre une erreur dans l'enregistrement des résultats.

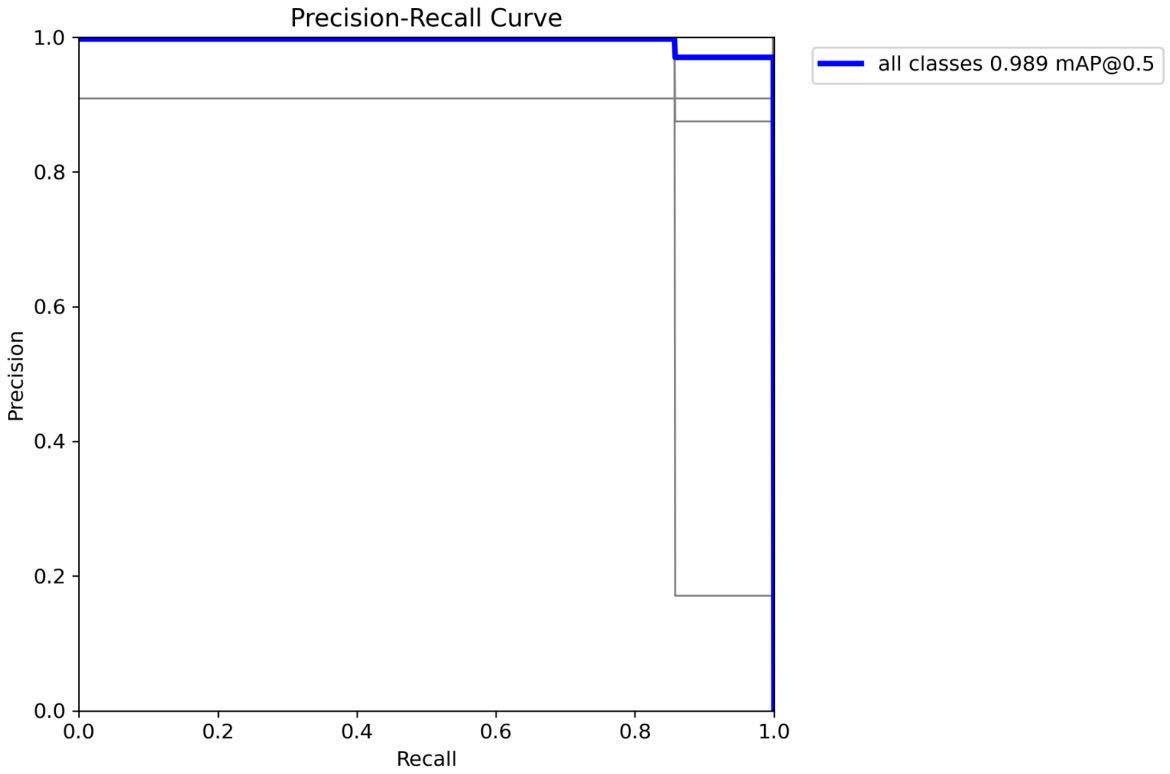


Figure 15 : Courbe de précision-rappel du modèle entrainé

La courbe figure 15 montre les résultats de précision selon le seuil de rappel choisi. On voit que la pluspart des labels ont 98% de précision à un rappel de 0,5 et commencent 0 chuter vers un rappel de 0,85. Certaines lignes grises se démarquent du reste en bleu car ce sont les tuiles avec moins de labels dans le dataset. Ils ont donc besoin de plus d'instances afin de relever leur score de précision.

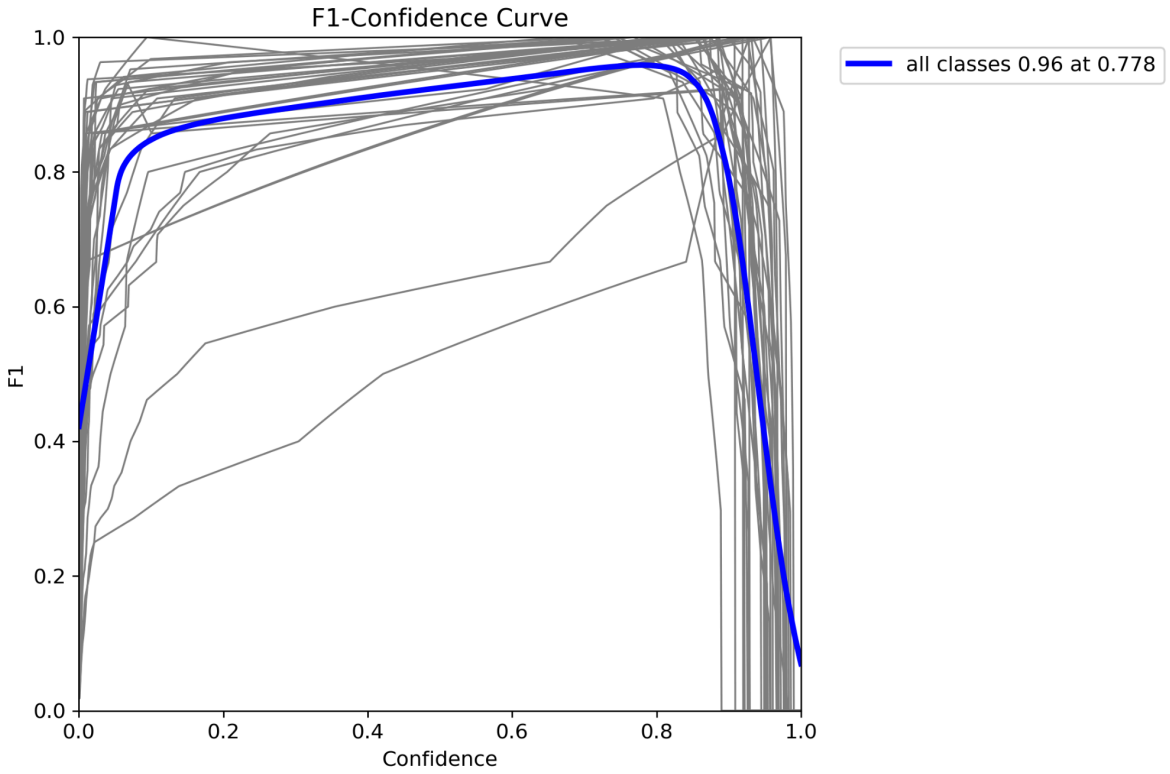


Figure 16 : Courbe de la moyenne de précision-rappel par rapport à un seuil de confiance

Le F1 de la figure 16 représente la moyenne entre la précision et le rappel. La courbe compare cette moyenne à un seuil de confiance, ce qui nous permet d'avoir par exemple ici : au seuil de confiance de 0.778, nous somme sûr à 96% d'avoir le meilleur résultat de prédiction possible avec le moins d'erreur de détection.



Figure 17 : Batch des images Val avec leur labels qu'on avait assigné



Figure 18 : Batch des images Val avec les labels que le modèle prédit à la fin de l'entrainement

Avec les figures 17 et 18, nous pouvons les comparer afin de vérifier que les résultats prédits par le modèle correspondent bien aux vrai labels. Cela permet de valider visuellement la précision du modèle.

On peut voir par exemple que même si sur la figure 18 certains labels sont cachés par d'autres, les couleurs des rectangles correspondent bien à leur images sur la figure 17 ce qui nous confirme que nous avons une très bonne précision.

L'objectif d'environ 90% de précision sur la majeur partie des tuiles qu'on avait décrit dans le cahier des charges est donc validé.

2.8 Développement de la logique du calcul des scores

Une fois que les tuiles ont été reconnues, on doit pouvoir calculer le score de la main par rapport aux règles spécifiques du Riichi Mahjong.

Au Riichi, le score se calcul par des Han et des Fu. Les Han sont les indicateurs principaux de score car ils influent le plus sur le score final. Les Fu sont des indicateurs secondaires qui sont utilisé lorsque la main est en dessous de 5 Han qui correspond au seuil minimum des scores normalisés avec des noms :

- 5 Han = Mangan
- 6-7 Han = Haneman
- 8-9-10 Han = Baiman
- 11-12 Han = Sanbaiman
- 13+ Han = Yakuman

De plus, si on gagne une main en tant que Dealer (Oya) on multiplie le score de la main par 1.5x.
Deux types de fin de manche sont possible :

- Ron = un autre joueur vous donne la tuile gagnante, dans ce cas la, il devra payer l'intégralité du score de la main gagnante.
- Tsumo = on pioche notre propre tuile gagnante. Deux cas de Tsumo sont possible.
  Le cas ou on n'est pas le Dealer, on divise le score par deux dans un premier temps, le Dealer de la manche paye cette moitié tandis que l'autre moitié sera divisé entre les deux autres joueurs.
  Le cas ou on est le Dealer, le score final sera divisé entre les trois joueurs équitablement.

Tous les paramètres suivants doivent donc être pris en compte par le système :

- Structure de la main (les groupes de tuiles, les pairs, les attentes de tuiles gagnantes)
- La combinaison de la main qui correspondent à des Yaku ou des Yakuman. C'est des équivalents aux mains de poker mais il y a 42 combinaisons différentes possible.
- Si la main est ouverte ou non, car cela influe sur l'obtention ou la valeur que certains Yaku rapportent.
- Les Dora, des tuiles qui correspondent à des Han bonus qui changent chaque partie. A part les 5 rouges qui correspondent toujours à 1 Dora donc 1 Han.
- La position du joueur, donc son vent actuel qui influe sur le Yaku appelé Yakuhai qui donne 1 Han si on a un brelan ou carré de notre vent.
- Le vent dominant qui fonctionne comme la position du joueurs mais qui change qu'une fois un tour de table complet se fait. Il permet a n'importe quel joueur d'avoir ce 1 Han si ils ont le brelan ou le carré de ce vent.
- La condition de victoire (Ron, Tsumo)
- Les bonus situationnels tels que :
    - le Riichi qui revient à déclarer aux autres joueurs que notre main est prête et attend une dernière tuile pour gagner. Cela permet de rapporter au joueur le déclarant 1 Han, mais l'empêche de changer sa main jusqu'à la fin de la manche.
    - l'Ippatsu qui survient pendant le premier tour de table après une déclaration de Riichi et qui rapporte 1 Han en plus si le joueur gagne pendant ce tour.
    - Les Ura Dora qui correspondent à des Dora bonus qu'on ne peut obtenir que si on gagne une main avec un riichi.

  D'autres bonus existent encore mais ceux là représente les plus importants.

L'ordre de calcul sera donc la suivante :

1. Détermination des Yaku accomplis par la main
2. Calcul des Han
3. Calcul des Fu

4. Application des plafonds de score et du score final

De plus, l'objectif étant d'aider les débutants, un détail des calcul sera fait afin de leur permettre de comprendre et de leur expliquer les résultats obtenus.
Cela permet donc d'être un outil d'apprentissage en plus de rendre le Riichi Mahjong plus accessible.



Figure 19 : Screenshot d'un résultat de calcul avec détail sur chaque étape

On voit bien que les Dora et le brelan de Haku sont bien pris en compte pour atteindre 3 Han. De plus avec 30 Fu et en non Dealer pour un Ron, on confirme avec le tableau Figure 20 que le score calculé correspond bien à 3900.

<table>
<tr><th colspan="4">Dealer</th><th>Ron</th><th colspan="4">Non-dealer</th></tr>
<tr><th>4 han</th><th>3 han</th><th>2 han</th><th>1 han</th><th>(Tsumo)</th><th>1 han</th><th>2 han</th><th>3 han</th><th>4 han</th></tr>
<tr><td>N/A
(2600)</td><td>N/A
(1300)</td><td>N/A
(700)</td><td rowspan="2">N/A</td><td>20 fu</td><td rowspan="2">N/A</td><td>N/A
(400/700)</td><td>N/A
(700/1300)</td><td>N/A
(1300/2600)</td></tr>
<tr><td>9600
(3200)</td><td>4800
(1600)</td><td>2400
(N/A)</td><td>25 fu</td><td>1600
(N/A)</td><td>3200
(800/1600)</td><td>6400
(1600/3200)</td></tr>
<tr><td>11600
(3900)</td><td>5800
(2000)</td><td>2900
(1000)</td><td>1500
(500)</td><td>30 fu</td><td>1000
(300/500)</td><td>2000
(500/1000)</td><td>3900
(1000/2000)</td><td>7700
(2000/3900)</td></tr>
<tr><td rowspan="5">12,000
(4,000)</td><td>7700
(2600)</td><td>3900
(1300)</td><td>2000
(700)</td><td>40 fu</td><td>1300
(400/700)</td><td>2600
(700/1300)</td><td>5200
(1300/2600)</td><td rowspan="5">8,000
(2,000 / 4,000)</td></tr>
<tr><td>9600
(3200)</td><td>4800
(1600)</td><td>2400
(800)</td><td>50 fu</td><td>1600
(400/800)</td><td>3200
(800/1600)</td><td>6400
(1600/3200)</td></tr>
<tr><td>11600
(3900)</td><td>5800
(2000)</td><td>2900
(1000)</td><td>60 fu</td><td>2000
(500/1000)</td><td>3900
(1000/2000)</td><td>7700
(2000/3900)</td></tr>
<tr><td>6800
(2300)</td><td>3400
(1200)</td><td>70 fu</td><td>2300
(600/1200)</td><td>4500
(1200/2300)</td></tr>
<tr><td>5 Han
Mangan
[満貫]</td></tr>
<tr><td colspan="4">18,000
(6,000)</td><td>6 - 7 Han
Haneman
[跳満]</td><td colspan="4">12,000
(3,000/6,000)</td></tr>
<tr><td colspan="4">24,000
(8,000)</td><td>8 - 10 Han
Baiman
[倍満]</td><td colspan="4">16,000
(4,000/8,000)</td></tr>
<tr><td colspan="4">36,000
(12,000)</td><td>11 - 12 Han
Sanbaiman
[三倍満]</td><td colspan="4">24,000
(6,000/12,000)</td></tr>
<tr><td colspan="4">48,000
(16,000)</td><td>12 < Han
Yakuman
[役満]</td><td colspan="4">32,000
(8,000/16,000)</td></tr>
</table>

Figure 20 : Tableau récapitulatif des scores finaux selon le nombre de Han et de Fu

Partie 3 – Mise en œuvre, gestion de projet et analyse critique

3.1 Méthodologie de gestion de projet

La réalisation du projet Riichi Tool a nécessité une organisation rigoureuse afin d'assurer la cohérence entre les objectifs techniques, les contraintes de temps ainsi que les ressources disponibles. Dans ce contexte, une méthodologie Agile a été retenue, en combinant les approches Scrum et Kanban.

Ce choix se justifie par la nature évolutive du projet, notamment pour la partie intelligence artificielle qui nécessite de nombreuses phases de tests, d'ajustements et d'amélioration continue. L'approche Agile permet de travailler par itérations courtes tout en conservant une forte capacité d'adaptation face aux imprévus techniques.

La méthode Scrum a permis de structurer le projet en plusieurs sprints d'une durée moyenne de deux semaines. Chaque sprint possédait des objectifs précis définis lors du Sprint Planning. Certains sprints étaient consacrés à la constitution du dataset et à l'annotation des images, d'autres à l'entraînement du modèle YOLO, au développement du backend Flask, à l'intégration de la base de données MySQL ou au développement du frontend.

Des points de suivi réguliers permettaient de mesurer l'avancement du projet, d'identifier rapidement les blocages techniques et d'ajuster les priorités si nécessaire. À la fin de chaque sprint, une phase de vérification permettait d'évaluer les résultats obtenus et d'améliorer l'organisation du sprint suivant.

En complément, la méthode Kanban a été utilisée pour visualiser l'avancement global du projet à l'aide d'un tableau organisé en plusieurs colonnes : “À faire”, “En cours”, “En test” et “Terminé”. Cette approche permet une meilleure gestion des priorités et une vision claire de la progression globale.

L’utilisation conjointe de Scrum et Kanban a permis d’assurer à la fois une bonne structuration du projet et une grande flexibilité dans la gestion quotidienne des tâches.

3.2 Organisation de l’équipe et répartition des rôles

L’équipe est répartie en 4 rôles différents : le Product Owner, le backend développeur, le frontend développeur, l’ingénieur IA.

Le rôle de product owner consistait à définir les objectifs, suivre l’avancée du projet à travers les sprints et vérifier que les objectifs du cahier des charges soient respectés.

Le rôle de développeur backend concernait la mise en place du serveur Flask, la création des routes API ainsi que la communication avec MySQL via MySQL Connector. Il s’est aussi occupé de développer le calculateur de score et de la création et de la sécurisation des bases de données MySQL.

Le rôle de développeur frontend portait sur la conception de l’interface utilisateur en HTML, CSS et JavaScript, avec pour objectif de proposer une interface simple, rapide et facilement testable.

Le rôle de Data Scientist consistait à constituer le dataset, annoter les images avec LabelImg, entraîner le modèle YOLO, tester les performances de détection et améliorer progressivement la précision du système.

Enfin, le rôle de testeur sera assumé par tout les membres du projet afin de valider le bon fonctionnement global du projet, d’identifier les erreurs de détection, de vérifier les calculs de score et de tester l’expérience utilisateur.

3.3 Planification du projet

Grâce à la méthode agile, un découpage de projet en plusieurs parties courtes devient possible et permet une adaptation simple et efficace après chaque sprint.

Les différentes phases du projet sont les suivantes :

- Première phase : analyse, étude du marché et du besoin. C'est aussi l'étape de définition du cahier des charges et des objectifs du projet.
- Deuxième phase : préparation des outils de développement et mise en place des parties principaux comme l'intégration de YOLOv11, le Flask et la page principale du Frontend.
- Troisième phase : entraînement du modèle YOLO ainsi que les tests de détection fait avec le Frontend qui est relié au flask. Début de l'implémentation du calculateur de score par rapport aux informations recus du Frontend.
- Quatrième phase : Fin de l'implémentation du calculateur. Mise en place des base de données et renforcement du modèle avec de nouvelles images étiquettés. Ajouts des pages d'historique des mains et de statistiques.
- Cinquième phase : Tests complets des systèmes implémentés ainsi que l'amélioration de l'ergonomie, optimisation et réglages de bugs découverts.

Cette organisation progressive a permis de limiter les blocages techniques et de sécuriser chaque étape avant le passage à la suivante tout en permettant à tout les pôles d'avancer en même temps.

3.4 Estimation des coûts et budget prévisionnel

En estimant une durée de développement initiale de trois à quatre mois pour obtenir un MVP fonctionnel, le budget humain peut être évalué de la manière suivante :

- Product Owner : 3 500 € / mois
- Développeur Backend : 4 000 € / mois
- Développeur Frontend : 3 500 € / mois
- Ingénieur IA : 4 000 € / mois

Le coût mensuel total estimé s'élève ainsi à environ 15 000 €.
Pour une période de 3 mois nous arrivons à environ 45 000 € pour les salires.

De plus, il faudra prendre en compte les coûts techniques tels que :

- L’hébergement du backend, du serveur web et de la base de données MySQL.
- Nom de domaine.
- Maintenance technique et sécurisation des données.
- Licence YOLO commerciale

On peut estimer ces coûts techniques pour un site de ce type entre 300 € et 800 €. Cela nous donnerai donc un total d’environ 50 000 € pour une première version public de Riichi Tool avec 6 mois de coûts techniques inclus.

Cet investissement reste cohérent pour une application spécialisée à forte valeur ajoutée, notamment sur un marché de niche comme celui du Riichi Mahjong compétitif. La rentabilité pourrait être assurée à moyen terme grâce à un modèle économique basé sur une version premium, des fonctionnalités avancées de statistiques, un abonnement utilisateur ou encore des fonctionnalités sociales et compétitives payantes.

3.5 Analyse des risques

Comme tout projet intégrant de l’intelligence artificielle et de la gestion de données, Riichi Tool présente plusieurs risques qu’il est nécessaire d’anticiper.
Le premier risque concerne la précision du modèle de détection. Avec un dataset limité, certaines erreurs de reconnaissance peuvent apparaître, notamment sur des tuiles similaires ou dans des conditions de faible luminosité.

Le deuxième risque concerne la complexité des règles du Riichi Mahjong. Une erreur dans la logique du calculateur pourrait produire un score incorrect, ce qui nuirait fortement à la crédibilité de l’application.

Le troisième risque concerne l’accessibilité utilisateur. Si l’interface est trop complexe ou si le processus de scan demande trop de corrections manuelles, les joueurs pourraient abandonner l’outil.

Le quatrième risque concerne la sécurité des données et la conformité au RGPD. L'enregistrement des comptes utilisateurs, des statistiques et de l'historique des parties nécessite une gestion rigoureuse de la protection des données personnelles.

Enfin, le projet présente également un risque de concurrence, puisque certaines applications similaires existent déjà sur le marché, notamment sur iOS.

Pour limiter ces risques, plusieurs solutions sont mises en place : amélioration progressive du dataset, vérification approfondie des règles de calcul avec un plus grand nombre de test de détection et de calculs en conditions réelles, sécurisation de la base de données et différenciation fonctionnelle par l'ajout de statistiques avancées et d'un système de progression.

3.6 Prototype et faisabilité technique

Un prototype fonctionnel complet a été réalisé et permet de valider la faisabilité technique du projet Riichi Tool. Ce prototype répond aux principales attentes fonctionnelles définies dans le cahier des charges de la section 2.3.

Faisabilité du système de reconnaissance visuelle

Le module de détection des tuiles repose sur le modèle YOLOv11 Small, entraîné sur un dataset d'environ 80 images annotées manuellement. Les résultats obtenus à l'issue de l'entraînement confirment la viabilité de cette approche pour la reconnaissance de tuiles de Riichi Mahjong. La courbe de précision-rappel présentée en Figure 15 indique un mAP50 de 98,9% toutes classes confondues, ce qui dépasse l'objectif de 90% fixé dans le cahier des charges. La courbe F1 en Figure 16 montre quant à elle un score F1 de 0,96 au seuil de confiance optimal de 0,778, confirmant un bon équilibre entre précision et rappel.

La comparaison visuelle entre les labels attendus (Figure 17) et les labels prédits par le modèle (Figure 18) valide visuellement ces résultats. Sur les images de validation, les tuiles sont reconnues avec une grande fiabilité, y compris dans des conditions variées de luminosité et d'angle. La principale limite identifiée concerne la tuile Haku (dragon blanc), dont l'absence de motif coloré complexifie la reconnaissance sur la

matrice de confusion, bien qu'elle soit correctement détectée en conditions réelles comme le montre la Figure 14.

Faisabilité du moteur de calcul des scores

Le moteur de calcul des scores, développé en Python, prend en charge l'ensemble des règles standard du Riichi Mahjong japonais. Il couvre la détection des 42 combinaisons de Yaku possibles, le calcul des Han et des Fu, l'application des paliers de score normalisés (Mangan, Haneman, Baiman, Sanbaiman, Yakuman), ainsi que les cas particuliers comme les double Yakuman. Les bonus situationnels tels que les bâtons Honba sont également pris en compte. La Figure 19 illustre un exemple de résultat détaillé produit par le système, permettant de vérifier la cohérence du calcul par rapport au tableau de référence présenté en Figure 20.

Faisabilité des fonctionnalités utilisateur

Le prototype intègre un système d'authentification complet avec inscription, connexion, déconnexion et suppression de compte, conforme aux exigences du RGPD. Un système de gestion de parties permet de regrouper plusieurs mains au sein d'une même session de jeu, avec suivi de l'évolution des scores de chaque joueur et affichage d'un graphique récapitulatif en fin de partie. L'historique des mains enregistrées est consultable et filtrable, et des statistiques globales sur les performances du joueur sont disponibles.

L'ensemble de ces fonctionnalités est accessible depuis un navigateur web standard sans installation requise, ce qui correspond bien à la contrainte d'accessibilité définie dans le cahier des charges.

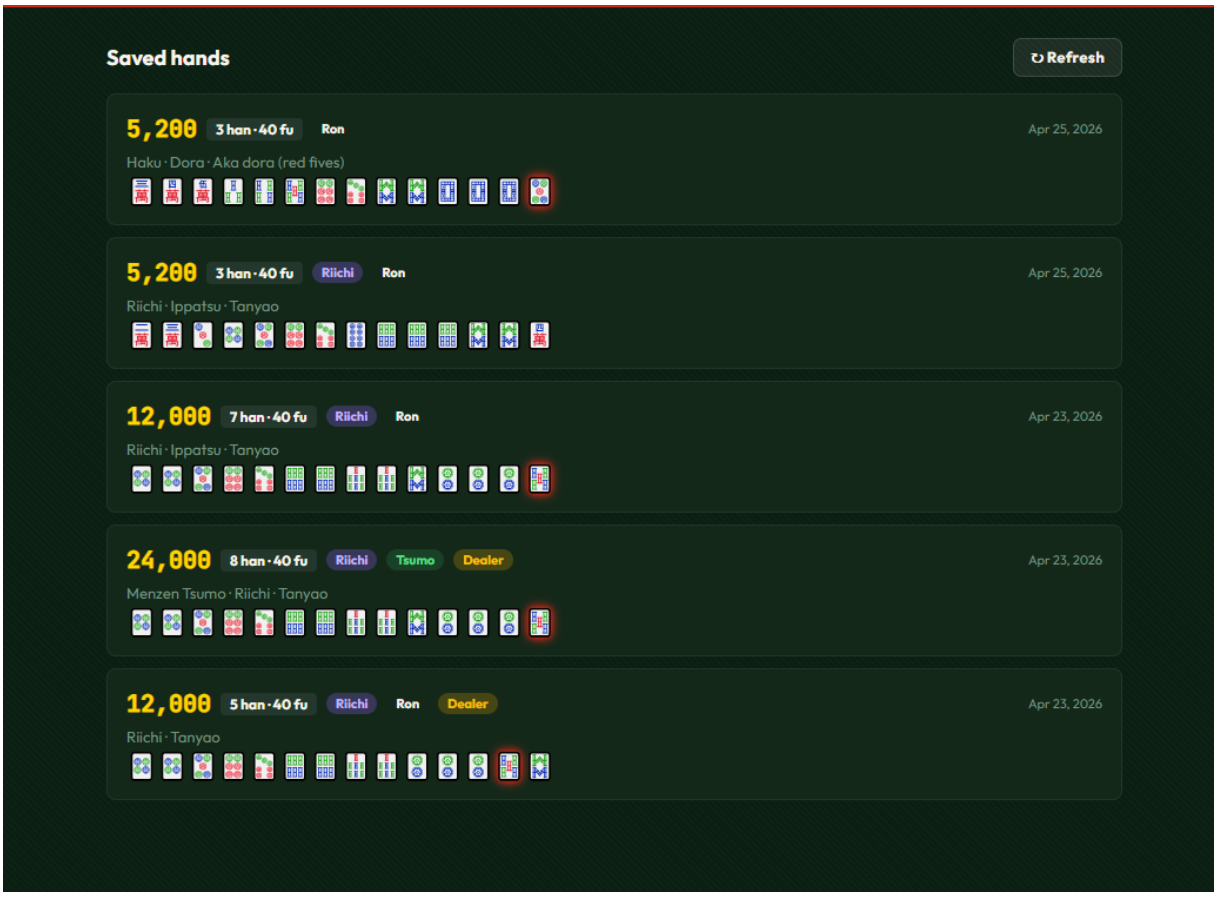


Figure 21 : Page d’historique des mains du joueurs



Figure 22 : Page de statistiques accessible sur le site

Bilan de faisabilité

Le prototype développé démontre que la combinaison d'un modèle de vision par ordinateur, d'un moteur de calcul de scores et d'une application web constitue une solution techniquement viable pour répondre à la problématique posée. Les objectifs principaux du cahier des charges sont atteints, et le prototype constitue une base solide pour les évolutions futures décrites en section 3.8.

3.7 Analyse critique et limites du projet

Afin d'analyser et d'évaluer le projet Riichi Tool, une analyse SWOT à été réalisée.



Figure 23 : Matrice SWOT de Riichi Tool

Analyse des forces

Le principal atout du projet réside dans la précision du système de reconnaissance visuelle, avec un mAP50 de 98,9% qui dépasse l'objectif initial de 90% fixé dans le cahier des charges. Cette performance démontre l'efficacité de l'apprentissage avec le modèle YOLOv11. Par ailleurs, le moteur de calcul des scores qui couvre l'intégralité des cas définis par les règles officielles du Riichi Mahjong japonais, constitue une preuve solide de l'efficacité de l'outil. Son accessibilité depuis un navigateur est aussi une force car les utilisateurs n'auront pas besoin de télécharger une application et pourront y accéder de n'importe quel endroit.

Analyse des faiblesses

La principale limite du projet est la taille du dataset d'entraînement. Avec un seul design de tuiles pour faire l'entraînement, le modèle aura du mal à prédire les tuiles des sets avec des design différents de celui utilisé lors de la constitution du dataset. De plus, l'inférence YOLO étant réalisée sur CPU, le temps de traitement peut varier en fonction des capacités matérielles du serveur, ce qui peut impacter l'expérience utilisateur. Enfin, l'interface web, bien qu'accessible depuis mobile, n'a pas été optimisée pour une utilisation sur celle-ci.

Analyse des opportunités

L'étude concurrentielle réalisée en section 2.2 a mis en évidence un manque réel de solutions combinant reconnaissance visuelle, calcul de score complet et historique des parties, notamment sur les plateformes Android et web. Ce positionnement représente une opportunité significative, d'autant plus que la communauté de joueurs de Riichi Mahjong est en croissance dans le monde.

Analyse des menaces

La principale menace identifiée est d'ordre légal. Le modèle YOLOv11 est distribué sous licence AGPL-3.0, qui impose de rendre le code source de l'application public en cas d'utilisation commerciale. Sans cela, une licence commerciale auprès d'Ultralytics serait nécessaire, représentant un coût estimé entre 500 et 2 000 euros par an selon la taille de la structure. Par ailleurs, la dépendance à un seul set de tuiles physiques constitue une limite à la généralisation du modèle, qui nécessiterait un réentraînement complet pour supporter de nouveaux designs de tuiles.

3.8 Perspectives d'évolution

Plusieurs améliorations possibles ont été identifiés durant le développement du projet :

- Renforcement du modèle avec l'ajout de plus de photos sous des conditions variées et des designs différents de tuiles.
- Migration du Frontend vers React afin d'avoir une meilleure gestion de l'application, une expérience plus fluide et une architecture plus adapté à une version mobile.

- Changement de modèle de YOLOv11 afin d'utiliser un GPU pour une meilleure précision et inférence.
- Migration de MySQL vers PostgreSQL pour avoir une gestion plus robuste des base de données.
- Système de recherche d'utilisateurs, et de gamification pour augmenter l'engagement des utilisateurs sur le long terme.
- Ajouts d'autres règles de Mahjong afin d'élargir la cible d'utilisateurs potentiels et donc de clients.
- Support multilingue afin d'offrir l'accessibilité du site à des utilisateurs internationaux.
- Enfin, une version en application mobile pour Android et iOS afin de pouvoir avoir une version utilisable partout.

Conclusion

La problématique posée en introduction soulevait la question de la faisabilité d'une application intelligente capable d'automatiser la reconnaissance et le calcul des scores dans le Riichi Mahjong. Le développement du prototype Riichi Tool permet de répondre à cette problématique. La combinaison d'un modèle de vision par ordinateur et d'un calculateur de score constitue une approche viable, y compris avec des ressources matérielles et un volume de données limités.

Ce projet a cependant mis en lumière que la performance d'un système d'intelligence artificielle ne repose pas uniquement sur l'algorithme choisi, mais autant sur la qualité et la diversité des données d'entraînement. C'est sans doute la leçon la plus importante de ce travail : un modèle bien entraîné sur peu de données soigneusement annotées peut surpasser des solutions disposant de davantage de moyens mais d'une approche moins rigoureuse.

Au-delà des aspects techniques, ce projet illustre plus largement le potentiel encore sous-exploité de l'intelligence artificielle appliquée aux jeux traditionnels. Là où des domaines comme les échecs ou le Go ont fait l'objet de recherches approfondies depuis des décennies, le Mahjong reste un terrain relativement vierge, notamment en ce qui concerne les outils à destination des joueurs non professionnels. Riichi Tool s'inscrit dans cette perspective en proposant une première réponse concrète à ce manque.

Les fondations posées au cours de ce travail offrent une base solide pour parvenir à atteindre un version complète dans l’avenir proche.

Utilisation de l’intelligence artificielle

L’Intelligence Artificielle ChatGPT à été utilisé comme aide afin de corriger et reformuler différents passages de ce mémoire. Les idées, analyses et choix ont été fait par un travail et des recherches personnelles.

Bibliographie

Applications et sites web analysés :

- **Mahjong Camera** - *App Store* : https://apps.apple.com/us/app/mahjong-camera/id6475007399
- **Jong Camera** - *Google Play Store* : https://play.google.com/store/apps/details?id=jp.gr.java_conf.hi_ragi.jongcamera
- **Riichi Calc** - *Google Play Store* : https://play.google.com/store/apps/details?id=ric.ov.RiichiCalc
- **Mahjong Calc Livewing** - *Site web* : https://mahjong-calc.livewing.net/
- **Agari** - *Site web* : https://agari.org/

Technologies et outils utilisés :

- **Ultralytics** - *Documentation officielle YOLOv11* : https://docs.ultralytics.com/
- **Figure 7 - Comparaison des performances YOLOv8 - YOLOv11** : https://docs.ultralytics.com/compare/yolov8-vs-yolo11/
- **YOLO vs Faster R-CNN** : https://pmc.ncbi.nlm.nih.gov/articles/PMC11157610/
- **Flask** - *Documentation officielle*: https://flask.palletsprojects.com/en/stable/
- **MySQL** - *Documentation officielle*: https://dev.mysql.com/doc/
- **LabelImg** - *Dépôt GitHub*: https://github.com/HumanSignal/labelImg

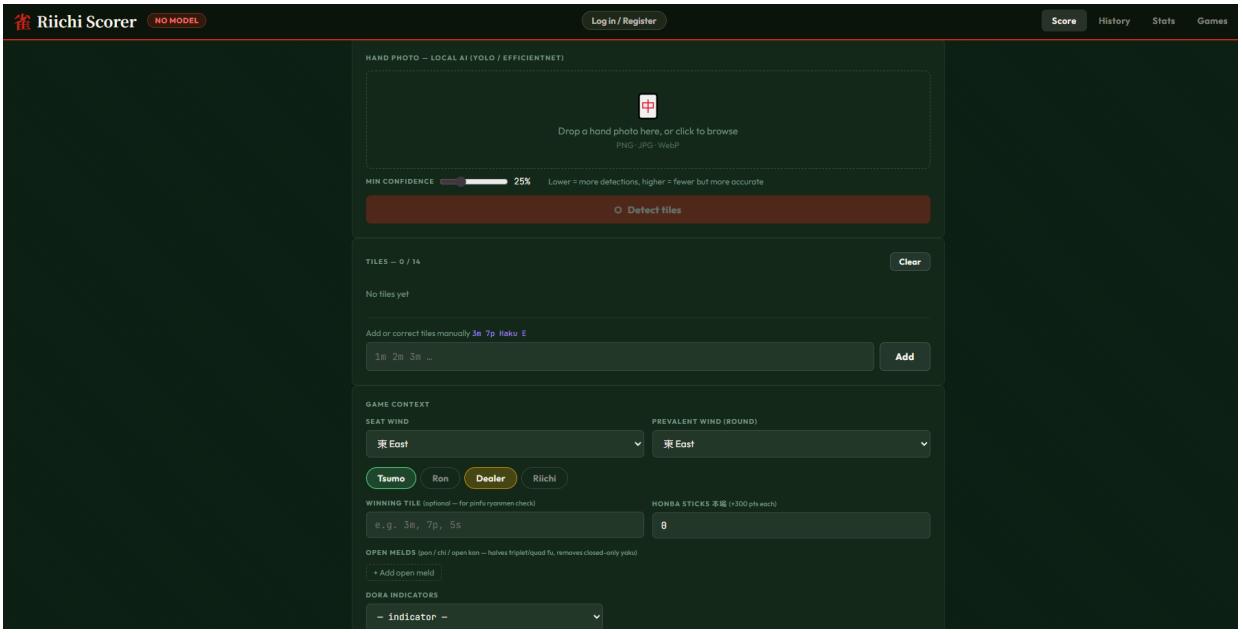
Concepts et références académiques :

- **AlphaGo** - *Article de Google Research DeepMind* : https://deepmind.google/research/alphago/
- **Suphx** - *Article de Microsoft Research - Suphx: Mastering Mahjong with Deep Reinforcement Learning, 2019, arxiv.org* : https://www.microsoft.com/en-us/research/publication/suphx-mastering-mahjong-with-deep-reinforcement-learning/
- **YOLO Original paper** - *You Only Look Once: Unified, Real-Time Object Detection, 2016* : https://arxiv.org/abs/1506.02640
- **RGPD** - *Texte officiel du réglement européen* : https://eur-lex.europa.eu/eli/reg/2016/679/oj?locale=fr
- **Scrum Guide** : https://scrumguides.org/

- **Figure 20 - Tableau de scoring Riichi Mahjong** : https://scoringtrainer.konbamwa.net

Annexes

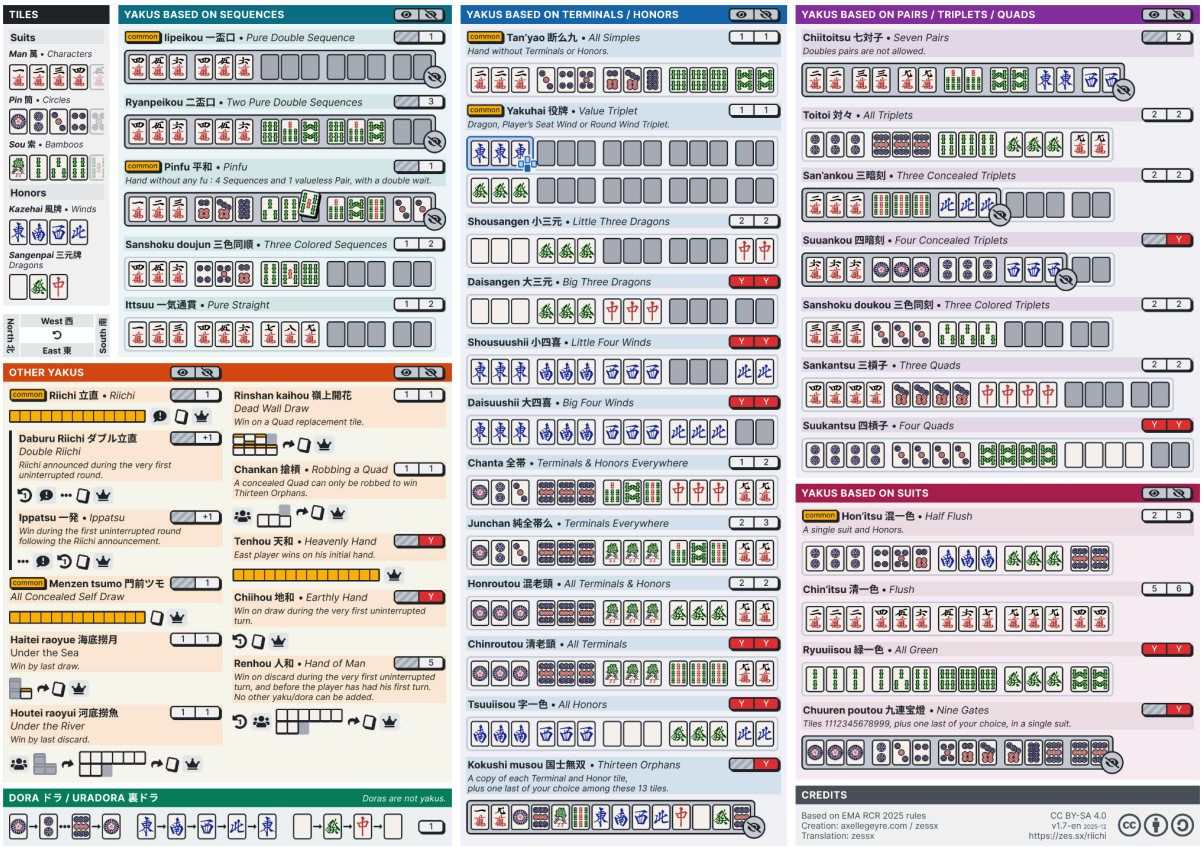
- Interface visuelle du détecteur et paramètres pour le calculateur de score



- Fichier CSV des résultats du dernier entraînement du modèle

	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O
1	epoch	time	train/box_loss	train/cls_loss	train/dfl_loss	metrics/precision(B)	metrics/recall(B)	metrics/mAP50(B)	metrics/mAP50-95(B)	val/box_loss	val/cls_loss	val/dfl_loss	lr/pg0	lr/pg1	lr/pg2
2	1	35.111	1.91525	5.16962	1.42864	0.00956	0.12184	0.00856	0.00411	1.63891	4.71022	1.32559	1.708e-05	1.708e-05	1.708e-05
3	2	70.509	1.54898	4.56569	1.21322	0.32485	0.1258	0.04869	0.03119	1.54576	4.17611	1.16487	3.62377e-05	3.62377e-05	3.62377e-05
4	3	102.363	1.29795	4.04463	1.05894	0.40226	0.1581	0.08842	0.06387	1.18638	3.86667	0.99429	5.50088e-05	5.50088e-05	5.50088e-05
5	4	134.732	1.19336	3.52215	1.00684	0.28572	0.24346	0.10852	0.08521	0.96917	4.00096	0.90548	7.33935e-05	7.33935e-05	7.33935e-05
6	5	167.02	1.08699	3.22702	0.97407	0.28501	0.35793	0.13406	0.11195	0.84224	3.99082	0.8722	9.13917e-05	9.13917e-05	9.13917e-05
7	6	199.366	0.97369	2.83949	0.93115	0.14848	0.5618	0.18471	0.15202	0.81345	3.69874	0.8649	0.000109003	0.000109003	0.000109003
8	7	231.47	0.95186	2.56069	0.94527	0.13918	0.5945	0.26108	0.20546	0.89127	3.53959	0.88818	0.000126229	0.000126229	0.000126229
9	8	264.297	0.9605	2.37139	0.94931	0.20044	0.60446	0.32828	0.26387	0.84675	3.22871	0.88434	0.000143067	0.000143067	0.000143067
10	9	296.639	0.85911	2.07182	0.91473	0.27178	0.62279	0.47057	0.38559	0.78721	2.39961	0.87656	0.000159519	0.000159519	0.000159519
11	10	328.259	0.84052	1.99159	0.92163	0.36058	0.63535	0.53411	0.43316	0.82556	2.0715	0.88619	0.000175585	0.000175585	0.000175585
12	11	359.668	0.82286	1.91681	0.89465	0.48569	0.60569	0.62132	0.50456	0.82435	1.85087	0.88468	0.000191264	0.000191264	0.000191264
13	12	391.282	0.83507	1.7859	0.90524	0.46862	0.64308	0.63987	0.51648	0.84211	1.7189	0.89106	0.000206557	0.000206557	0.000206557
14	13	422.624	0.80843	1.62183	0.90848	0.71667	0.57079	0.68918	0.55363	0.81775	1.65198	0.88693	0.000215013	0.000215013	0.000215013
15	14	453.865	0.75636	1.51263	0.88507	0.5288	0.72356	0.728	0.59095	0.81718	1.56862	0.8823	0.000212597	0.000212597	0.000212597
16	15	485.504	0.79336	1.46459	0.91296	0.61758	0.7316	0.78534	0.6344	0.82399	1.39495	0.8815	0.000210182	0.000210182	0.000210182
17	16	516.61	0.76098	1.30861	0.88514	0.76311	0.6738	0.79833	0.65597	0.76261	1.3001	0.86962	0.000207766	0.000207766	0.000207766
18	17	547.917	0.77256	1.32979	0.89997	0.69756	0.76659	0.81186	0.66975	0.73673	1.19295	0.86636	0.00020535	0.00020535	0.00020535
19	18	579.102	0.76907	1.18918	0.89188	0.68242	0.83586	0.83418	0.67488	0.7731	1.07775	0.87321	0.000202935	0.000202935	0.000202935
20	19	610.086	0.76058	1.15237	0.9058	0.85301	0.7766	0.8621	0.70944	0.74876	0.98319	0.86621	0.000200519	0.000200519	0.000200519
21	20	641.55	0.74894	1.12389	0.89231	0.80251	0.81942	0.90972	0.75639	0.72917	0.91682	0.85973	0.000198104	0.000198104	0.000198104
22	21	672.926	0.74596	1.05339	0.88248	0.78576	0.83811	0.91647	0.7574	0.74469	0.888	0.86113	0.000195688	0.000195688	0.000195688
23	22	703.907	0.71356	0.96659	0.87691	0.72705	0.93429	0.93315	0.76947	0.74226	0.84096	0.86456	0.000193272	0.000193272	0.000193272
24	23	735.103	0.74512	0.95716	0.88989	0.78521	0.93276	0.93693	0.74905	0.82459	0.79071	0.88736	0.000190857	0.000190857	0.000190857
25	24	766.455	0.74375	0.98917	0.88529	0.8482	0.92783	0.94758	0.76099	0.81524	0.74917	0.88706	0.000188441	0.000188441	0.000188441
26	25	797.517	0.68914	0.81831	0.86659	0.87767	0.93276	0.96879	0.78885	0.76827	0.69078	0.87487	0.000186026	0.000186026	0.000186026
27	26	828.892	0.70777	0.83484	0.88444	0.92621	0.92514	0.97671	0.79689	0.7479	0.65896	0.86605	0.00018361	0.00018361	0.00018361
28	27	860.173	0.73094	0.85385	0.8843	0.92289	0.91661	0.97803	0.78752	0.78165	0.65353	0.87195	0.000181194	0.000181194	0.000181194
29	28	891.505	0.71435	0.78925	0.87923	0.89423	0.94683	0.98349	0.78203	0.84877	0.65282	0.88603	0.000178779	0.000178779	0.000178779
30	29	923.011	0.68926	0.72539	0.87437	0.9118	0.94939	0.98648	0.79656	0.8105	0.62139	0.87816	0.000176363	0.000176363	0.000176363
31	30	953.9	0.67659	0.69765	0.8778	0.92888	0.94863	0.98614	0.81742	0.7163	0.58482	0.86024	0.000173948	0.000173948	0.000173948
32	31	985.28	0.6924	0.67768	0.88151	0.93908	0.95058	0.98662	0.82694	0.69273	0.56815	0.85736	0.000171532	0.000171532	0.000171532
33	32	1016.49	0.7015	0.71452	0.87429	0.94659	0.96382	0.98802	0.81202	0.72895	0.55741	0.86393	0.000169116	0.000169116	0.000169116
34	33	1047.72	0.68883	0.71083	0.88205	0.93616	0.97379	0.9883	0.81175	0.73558	0.54362	0.86274	0.000166701	0.000166701	0.000166701
35	34	1078.84	0.66869	0.68097	0.86448	0.93117	0.9646	0.98865	0.81588	0.72777	0.53178	0.85963	0.000164285	0.000164285	0.000164285
36	35	1110.09	0.65629	0.62289	0.86773	0.93139	0.95732	0.98894	0.82409	0.69965	0.52836	0.85365	0.00016187	0.00016187	0.00016187
37	36	1141.16	0.65176	0.60531	0.86441	0.92982	0.95949	0.98894	0.8241	0.70869	0.53298	0.85511	0.000159454	0.000159454	0.000159454
38	37	1172.15	0.66508	0.62089	0.86385	0.92394	0.9567	0.99006	0.81091	0.76903	0.54332	0.86868	0.000157038	0.000157038	0.000157038
39	38	1203.13	0.675	0.60906	0.87302	0.92261	0.95516	0.99018	0.81516	0.76053	0.53846	0.86767	0.000154623	0.000154623	0.000154623
40	39	1234.46	0.68169	0.61929	0.88091	0.93394	0.95314	0.97575	0.81678	0.6851	0.5193	0.85275	0.000152207	0.000152207	0.000152207
41	40	1265.5	0.6771	0.61992	0.86247	0.93228	0.9681	0.97505	0.82202	0.6737	0.50659	0.85141	0.000149792	0.000149792	0.000149792
42	41	1296.69	0.64111	0.56965	0.86729	0.9463	0.96345	0.97416	0.81314	0.69641	0.5052	0.85498	0.000147376	0.000147376	0.000147376
43	42	1327.92	0.66876	0.57785	0.86849	0.9482	0.97455	0.97535	0.79746	0.75768	0.50843	0.86708	0.00014496	0.00014496	0.00014496
44	43	1358.92	0.63862	0.57465	0.85577	0.94887	0.97426	0.98914	0.82023	0.73311	0.49539	0.86106	0.000142545	0.000142545	0.000142545
45	44	1390.15	0.60816	0.53019	0.84917	0.9405	0.97569	0.97424	0.82523	0.67682	0.48629	0.84868	0.000140129	0.000140129	0.000140129
46	45	1421.22	0.65317	0.55146	0.86494	0.92648	0.97598	0.98718	0.83757	0.64345	0.49711	0.8434	0.000137714	0.000137714	0.000137714
47	46	1452.27	0.66613	0.55694	0.86715	0.91108	0.97497	0.98628	0.83753	0.64956	0.5039	0.84301	0.000135298	0.000135298	0.000135298
48	47	1483.29	0.66859	0.57829	0.87283	0.90815	0.97102	0.98642	0.83368	0.67598	0.50588	0.84459	0.000132882	0.000132882	0.000132882
49	48	1514.22	0.6392	0.54695	0.8617	0.92671	0.96701	0.98792	0.8012	0.77669	0.52196	0.86338	0.000130467	0.000130467	0.000130467

- Liste des différents Yaku (combinaisons) dans le Riichi Mahjong : https://zes.sx/riichi/



- Set de Mahjong utilisé pour le dataset : https://www.amazon.co.jp/-/en/dp/B09MLZBNF1?ref=ppx_yo2ov_dt_b_fed_asin_title



Glossaire

- **Intelligence artificielle et apprentissage automatique**

Annotation : Processus consistant à étiqueter manuellement des données d'entraînement. Dans ce projet, l'annotation désigne le fait d'encadrer chaque tuile sur les photos et de lui assigner un label correspondant à l'aide de LabelImg.

Batch : Groupe d'images traité simultanément par le modèle lors d'une étape d'entraînement. Un epoch complet correspond au traitement de l'ensemble du dataset divisé en batches successifs.

Bounding box : Rectangle délimitant la position d'un objet détecté sur une image. Chaque tuile détectée par YOLO est entourée d'un bounding box associé à un label et un score de confiance.

Dataset : Ensemble de données utilisé pour entraîner et évaluer un modèle de machine learning. Dans ce projet, le dataset est constitué de photographies de mains de Riichi Mahjong annotées manuellement.

Deep Learning : Sous-domaine du machine learning reposant sur des réseaux de neurones artificiels à plusieurs couches. Utilisé dans ce projet pour la reconnaissance visuelle des tuiles.

Epoch : Cycle complet d'entraînement durant lequel le modèle parcourt l'intégralité du dataset une fois. Un entraînement se compose généralement de plusieurs dizaines ou centaines d'epochs.

F1 Score : Moyenne harmonique entre la précision et le rappel. Permet d'évaluer globalement la performance d'un modèle en tenant compte à la fois des faux positifs et des faux négatifs.

Inférence : Phase d'utilisation d'un modèle entraîné pour effectuer des prédictions sur de nouvelles données. Dans ce projet, l'inférence correspond à la détection des tuiles sur une photo soumise par l'utilisateur.

mAP50 : Mean Average Precision calculée à un seuil d'acceptation de 50%. Mesure la précision moyenne du modèle sur l'ensemble des classes en ne comptant comme correctes que les détections dont le bounding box recouvre la réalité à plus de 50%.

Matrice de confusion : Tableau représentant les prédictions du modèle par rapport aux vraies valeurs. Permet d'identifier les classes qui sont fréquemment confondues entre elles.

ONNX : Open Neural Network Exchange. Format standard d'exportation de modèles de machine learning permettant de les utiliser indépendamment du framework avec lequel ils ont été entraînés.

Overfitting : Phénomène de surapprentissage où le modèle apprend trop précisément les données d'entraînement et perd sa capacité de généralisation sur de nouvelles données inconnues.

Poids : Paramètres numériques internes d'un réseau de neurones, ajustés à chaque étape d'entraînement afin de minimiser l'erreur de prédiction.

Précision : Proportion de détections correctes parmi l'ensemble des détections effectuées par le modèle. Une précision élevée signifie peu de faux positifs.

Rappel : Proportion de tuiles réellement présentes sur l'image qui ont été détectées par le modèle. Un rappel élevé signifie peu de tuiles manquées.

Seuil de confiance : Valeur minimale de certitude requise pour qu'une détection soit acceptée. En dessous de ce seuil, la détection est ignorée.

- **Développement web et base de données**

Backend : Partie serveur de l'application, responsable de la logique métier, du traitement des données et de la communication avec la base de données. Développé avec Flask dans ce projet.

Flask : Micro-framework Python utilisé pour développer le backend de l'application. Léger et flexible, il permet de créer des API REST rapidement.

Foreign Key : Clé étrangère. Contrainte de base de données garantissant que la valeur d'un champ correspond bien à une valeur existante dans une autre table, assurant ainsi l'intégrité référentielle des données.

Frontend : Partie client de l'application, correspondant à l'interface utilisateur affichée dans le navigateur. Développé en HTML, CSS et JavaScript natif dans ce projet.

Hash : Résultat d'une fonction de chiffrement à sens unique appliquée à un mot de passe. Permet de stocker les mots de passe de manière sécurisée sans jamais conserver leur valeur originale.

MySQL : Système de gestion de base de données relationnelle open-source utilisé pour stocker les mains, les parties et les informations utilisateurs.

- **Gestion de projet**

Backlog : Liste priorisée des fonctionnalités et tâches à réaliser dans le cadre du projet. Géré par le Product Owner et mis à jour à chaque sprint.

Kanban : Méthode de gestion visuelle du flux de travail reposant sur un tableau organisé en colonnes représentant les différents états d'avancement des tâches.

Product Owner : Membre de l'équipe responsable de la définition des priorités du backlog et de la validation des fonctionnalités développées à chaque sprint.

Scrum : Cadre de travail Agile organisant le développement en itérations courtes appelées sprints, avec des cérémonies régulières de planification et de bilan.

Sprint : Itération de durée fixe, généralement deux semaines, à l'issue de laquelle un incrément fonctionnel du projet doit être livré.

- **RGPD et sécurité**

RGPD : Règlement Général sur la Protection des Données. Réglementation européenne encadrant la collecte, le traitement et la conservation des données personnelles des utilisateurs.

Droit à l'effacement : Droit accordé à tout utilisateur de demander la suppression de l'ensemble de ses données personnelles. Implémenté dans ce projet via la fonctionnalité de suppression de compte.

Minimisation des données : Principe du RGPD imposant de ne collecter que les données strictement nécessaires au fonctionnement du service.

- **Riichi Mahjong**

Aka Dora : Tuile 5 rouge présente en un exemplaire dans chaque famille. Compte toujours comme un Han bonus quel que soit l'indicateur Dora en cours.

Dealer / Oya : Joueur occupant la position Est en début de manche. Le dealer paie et reçoit des montants majorés par rapport aux autres joueurs.

Dora : Tuile bonus déterminée par un indicateur retourné en début de manche. Chaque tuile Dora dans la main gagnante rapporte un Han supplémentaire.

Fu : Unité de mesure secondaire du score, calculée à partir de la structure de la main. Utilisée uniquement lorsque la main est en dessous de cinq Han.

Han : Unité de mesure principale du score au Riichi Mahjong. Chaque Yaku accompli rapporte un certain nombre de Han qui déterminent le montant final gagné.

Honba : Bâtons de répétition accumulés lors des mains consécutives sans changement de dealer. Chaque Honba ajoute 300 points au gain de la main gagnante.

Ron : Victoire obtenue en réclamant la tuile défaussée par un adversaire. Le joueur ayant défaussé la tuile paye l'intégralité du score.

Tsumo : Victoire obtenue en piochant soi-même la tuile gagnante. Le paiement est réparti entre les trois autres joueurs.

Ura Dora : Tuiles bonus supplémentaires révélées uniquement en cas de victoire avec Riichi. Chaque Ura Dora dans la main rapporte un Han bonus.

Yakuman : Main d'exception rapportant un score fixe très élevé, indépendamment du calcul Han/Fu standard. Certaines mains particulièrement difficiles constituent un double Yakuman.

Yaku : Combinaison de tuiles reconnue par les règles du Riichi Mahjong et rapportant un certain nombre de Han. Une main gagnante doit obligatoirement contenir au moins un Yaku.